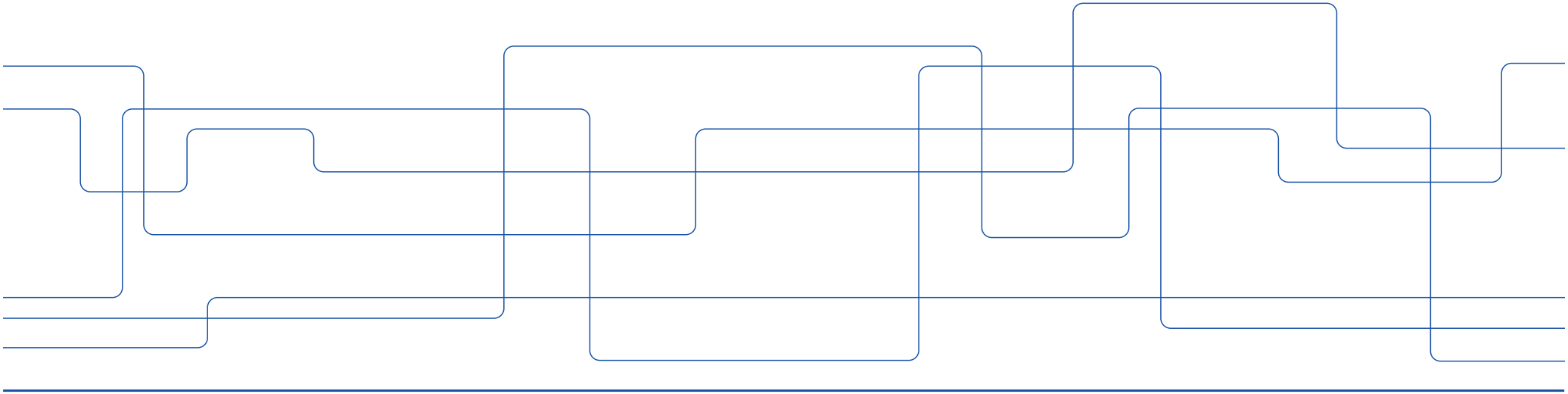


HI1024

Grundkurs i programmering

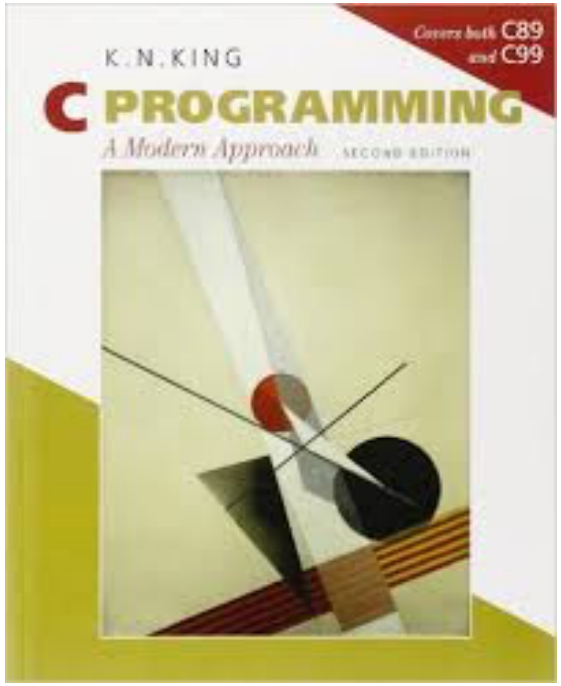
F9: Pekare!

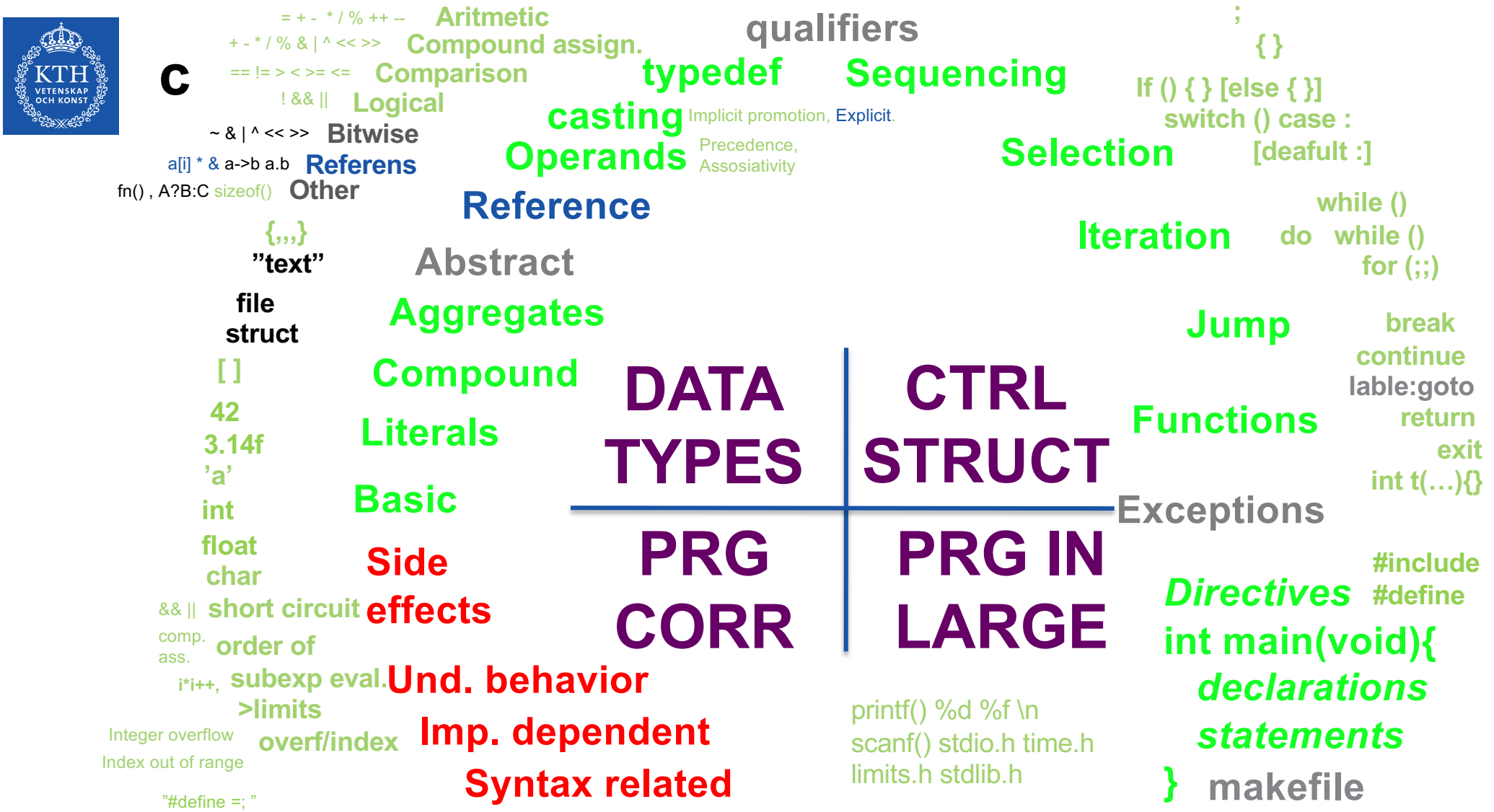


Innehåll!

King 11.1-4 & 12.3-4

- Repetition föreläsning #8
- Mer om Moduler
- Pekare
- Lab #2
- Övning




C
= + - * / % ++ --
Aritmetic
+ - * / % & | ^ << >>
Compound assign.
== != > < >= <=
Comparison
! && ||
Logical
~ & | ^ << >>
Bitwise
a[i] * & a->b a.b
Referens
fn() , A?B:C sizeof()
Other
{,,,}
”text”
file
struct
[ ]
42
3.14f
’a’
int
float
char
Basic
Literals
Compound
Aggregates
Abstract
Reference
qualifiers
typedef
Sequencing
casting
Implicit promotion, Explicit.
Operands
Precedence, Assosiativity
;
{ }
If () { } [else { }]
switch () case :
[deafult :]
Selection
while ()
do while ()
for (;;)
Iteration
Jump
break
continue
lable:goto
return
exit
int t(…){}
Functions
Exceptions
DATA TYPES
CTRL STRUCT
PRG CORR
PRG IN LARGE
#include
#define
Directives
int main(void){
declarations
statements
} makefile
printf() %d %f \n
scanf() stdio.h time.h
limits.h stdlib.h
&& || short circuit
comp. ass.
order of
i*i++,
subexp eval.
Side effects
Und. behavior
Imp. dependent
Syntax related
>limits
Integer overflow
Index out of range
overf/index
”#define =; ”
int/int
/*comment*/ or //

Fälla!

```
#include <stdio.h>
#define WEEKDAYS 7

void printBox(int aBox[]) {
    for (int i=0; i<sizeof(aBox)/sizeof(aBox[0]); i++)
        printf("Box[%d] : %d\n", i, aBox[i]);
}

int main(int argc, char **argv) {
    int box[WEEKDAYS]={2,1,1,1,0,0,1};
    printf("Matrix tester (MT)!\n");

    printBox(box);

    printf("MT Done, bye!\n");
    return 0;
}
```

2 / 1 = 2

```
Matrix tester (MT)!
Box[0] : 2
Box[1] : 1
MT Done, bye!
```

```
 PROGRAM
#0000: CODE
   *
 DATA VÄRDE      NAMN          TYP
#F000:   2    box[0]aBox[0]    int
#F001:   1    box[1]aBox[1]    int
#F002:   1    box[2]aBox[2]    int
#F003:   1    box[3]aBox[3]    int
#F004:   0    box[4]aBox[4]    int
#F005:   0    box[5]aBox[5]    int
#F006:   1    box[6]aBox[6]    int
#F007:
#F008:
#F009:
```

R

…en matris parameter är i själva verket enbart adressen till första elementet, därför behöves en parameter till (eller ett annat trick) för att tala om hur många element som faktiskt finns…

…i den här dator är en adress tydligen dubbelt så minneskrävande som en int!

…använda aldrig sizeof() för att beräkna hur många element som finns (om du inte är otroligt säker på din sak)!

Rekursion

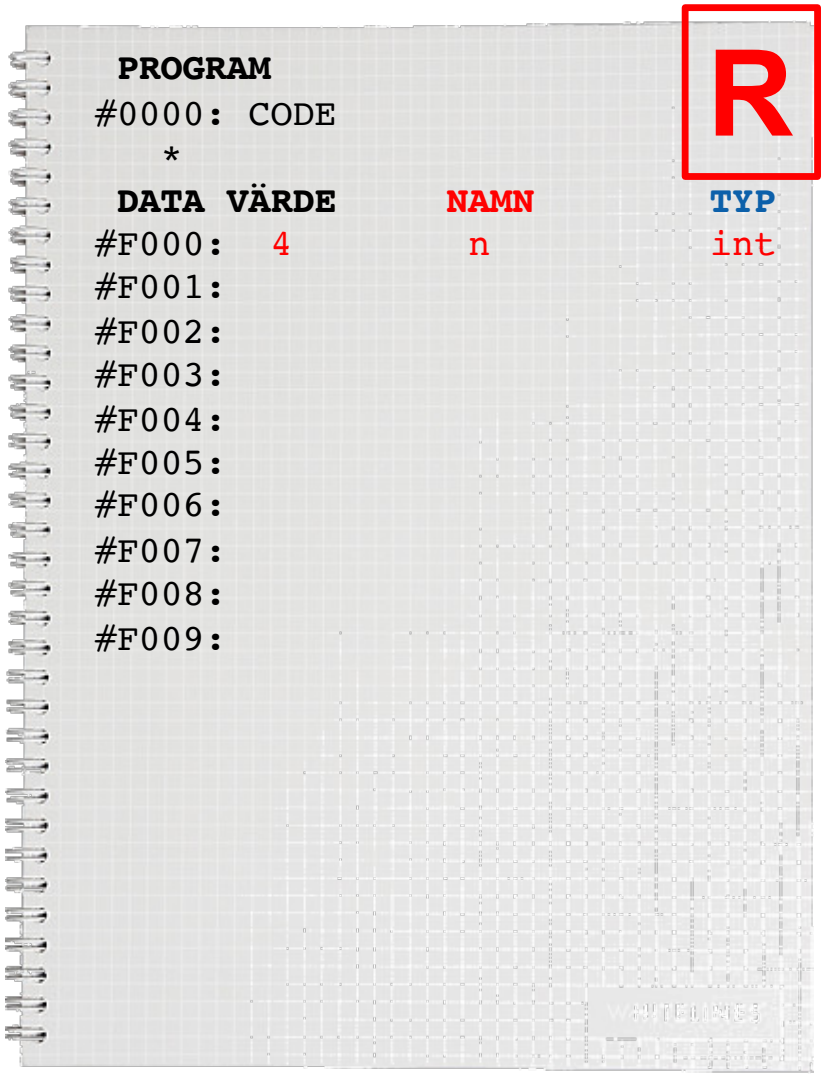
En funktion kan anropa sig själv, det sägs då vara ett rekursivt anrop!

T.ex. kan fakulteten av ett tal n lätt beräknas som:
n! = n * (n – 1)! Där n! = 1.

```
int fact(int n){
  if (n<=1) 4
    return 1;
  else
    return n * fact(n-1);
}
```

3

```
printf("4!=%d",fact(4));
```



Lokala vs Globala variabler

```c
/* Character 0.4 AC */
#include <stdio.h>
#define WEEKDAYS 7
#define DAYSLOTS 4

void printBox(int theBox[][WEEKDAYS], int rows);

int main(void){

    int w[DAYSLOTS][WEEKDAYS]={ {1,2,3,4,5,6,7},
                                {1,2,3,4,5,6,7},
                                {1,2,3,4,5,6,7},
                                {1,2,3,4,5,6,7} };

    printf("Matrix Tester (MT)!\n");

    printBox( w, DAYSLOTS );

    printf("MT Done.\n");
    return 0;
}

void printBox(int theBox[][WEEKDAYS], int rows) {
   for (int i=0; i<rows; i++)
        for (int j=0; j<WEEKDAYS; j++)
      printf("Box[%d,%d] : %d\n",i,j,theBox[i][j]);
}
```

Variabeln w sägs vara lokal därför att den deklarerats inuti en funktion!

En lokal variabel skapas varje gång omslutande funktion anropas under exekvering, och den glöms bort så snart funktionen når sitt slut!

M.a.o kan den hamna på olika ställen i minnet från gång till gång…

Automatic storage duration, Block scope (visability)

En lokal variabel ”syns” inte utanför sin funktion, värdet kan sändas som argument till en annan funktion!

Lokala vs Globala variabler

Variabeln w sägs vara global därför att den deklarerats utanför alla funk!

En global variabel skapas när koden passeras under exekvering där den är deklarerad, och den finns kvar så länge kod från samma fil exekveras.

En global variabel är synlig i hela filen från den punkt där den deklareras! Den kan dock skuggas av lokala variabler/parametrar.

Static storage duration, File scope (visability)

```c
/* Character 0.4 AC */
#include <stdio.h>
#define WEEKDAYS 7
#define DAYSLOTS 4

void printBox();

    int w[DAYSLOTS][WEEKDAYS]={ {1,2,3,4,5,6,7},
                                {1,2,3,4,5,6,7},
                                {1,2,3,4,5,6,7},
                                {1,2,3,4,5,6,7} };

int main(void){

    printf("Matrix Tester (MT)!\n");

    printBox( );

    printf("MT Done.\n");
    return 0;
}

void printBox() {
   for (int i=0; i<DAYSLOTS; i++)
        for (int j=0; j<WEEKDAYS; j++)
      printf("Box[%d,%d] : %d\n",i,j,w[i][j]);
}
```


Om en global variabel fått ett felaktigt värde är det mycket svårt att ta reda på vart, vi använder det inte i kursen!

Programutvecklingsmetodik

Den enkla metodik vi använt så här långt fungerar utmärkt om det är ett begränsat problem som vi ska skriva kod för, ett litet program eller en modul till ett större program…

…är problem svårare måste det först brytas ner till hanterbara mindre problem som vi nu kan lösa…

…hur det går till är stoff för flera års studier med det är definitivt en iterativ process där tanken är att nerlagt arbete successivt ska ge större och större insikt i hur problemet bör tacklas!

R

Algoritmer & Datastrukturer 0

1 Skriv ner proble-met så att ett barn förstår det!

Gör ett minimalt main() av dina anteckningar! 2

3 a..n Konstanter Variabler Uttryck & resultat!

Läs in riktiga värden! 4

Bottom up vs Top Down

Top Down, givet ett grundlig arbete med system arkitekturen, brukar vara bäst!

Javisst, en på-knapp behövs också…

Sen behövs det kanske någon form av styrning också...

Wow!!!! den måste ha en brutal motor!



Den behöver ett bra gränssnitt…

…som exponerar styrdatorns alla program…

…så att motorn körs på rätt sätt!

Top Down ger också något som går att testa/visa upp för satsningens intressenter tidigt!

I undantagsfall går det att bygga fast sig i en felaktig sys.ark, men då har någon fuskat när satsningen började!

Top Down



- Titta på problemet från ett användarperspektiv
- Skapa en systemarkitektur genom att dela upp problemet i allt mindre byggblock tills varje byggblock är överblickbart, om möjligt också frikopplat från andra byggblock (få beroenden) samt potentiellt återanvändbart!
- Implementera användargränssnittet, både för att möjliggöra enkel testning av underliggande byggblock allt eftersom de blir klara men även för att kunna ge en bild av vad som byggs för satsningen intressenter.
- Ett byggblock benämns ofta modul (eng: unit) och när systemarkitekturarbetet är klart bör det finnas en specifikation på huvuddelen/alla moduler som behövs inklusive exakt vilka funktioner de ska erbjuda (eng: interface specification).
- Med lite tur så finns kanske flertalet av modulerna redan då de utvecklats för andra satsningar, de som återstår måste utvecklas, och vissa kan kanske återanvändas senare.

Varje enskild modul kan därefter utvecklas enligt 0..4 metoden!



Best Practic



- En driven programmerar har i bakhuvudet vilka moduler som redan finns under systemarkitekturarbetet, kan en befintlig modul användas kan det spara både tid och pengar.
- **En modul ska om möjligt inte avslöja något om hur den skapar erbjudna tjänster, utan bara vilka det är, detta kallas abstraktion!**
- I kursen har vi redan skapat 2 återanvändbara moduler med 2 tjänster var:

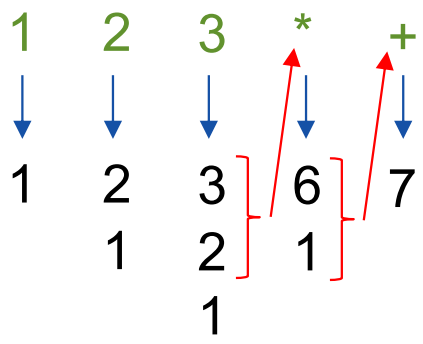
initRandomGenerator()
Återanvändbara slumpfunktioner!
rndNumber(52)

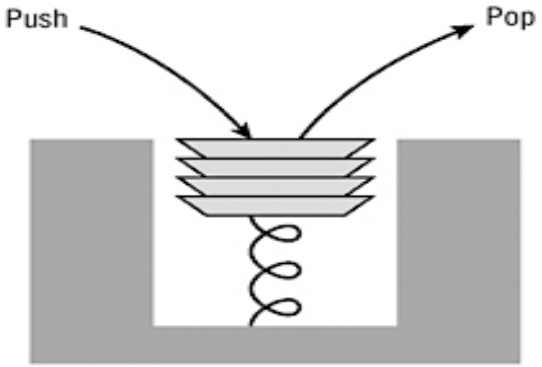
printCard(theCard)
Återanvändbara kortfunktioner!
getRank(theCard)

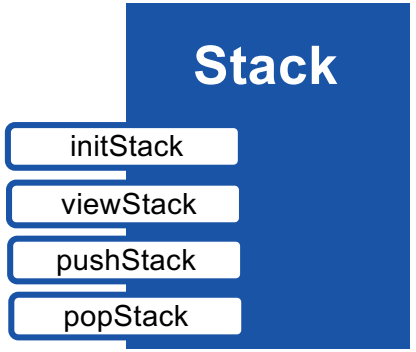
- Att samla på sig testade och beprövade moduler är nyckeln till framgång i denna bransch! Dags att träna på att skapa en till mycket användbar modul!

RPN miniräknare, Chapter10: P6

- Företaget du arbetar för tänker lansera ett litet miniräknarprogram där tal matas in med hjälp av den så kallade omvänd polsk notation (HP). Sättet är ursmart då det inte kräver parenteser eller lika med (=), se exempel…
- Arbetssättet för att lösa: 1 2 3 * + (= 7) är som följer:
- Tänk dig att du har en fjädrande behållare som ”talrikshållaren i matsalen”. Tal ”sparas”, om det däremot är en operation så tas de två översta talen från behållaren och operationen utförs innan svaret sparas i behållaren! M.a.o



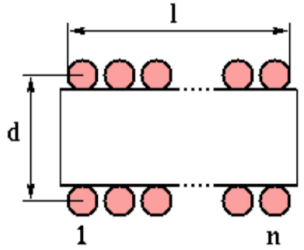




”Tallrikshållaren” kallas för en stack, tänk om vi haft en sådan…

Reflektioner övning #8

1. (4p) Enkla luftlindade spolar är en viktig komponent radiosammanhang. Formeln för en sådan spoles induktans kan beräknas med följande formel:



$L = (d^2n^2)/(l+0.45d)$ [uH], där d och l anges i **meter**.

Du ska skriva ett enkelt program som frågar efter spolens **diameter i mm**, spolens **längd i mm** samt **hur många varv** tråd spolen består av. Du ska därefter med hjälp av formeln beräkna spolens induktans. Svaret ska presenteras med **2 decimaler** och uttryckt **i enheten uH**. TIPS: Tänk på att int / int blir en int!

Programmet ska också innehålla möjligheten att göra en ny beräkning! Ditt program bör likna följande exempel!



```
Single layer coil inductans calcylator.
Enter diameter in mm ?10
Enter length in mm ?10
Enter number of turns ?10
Inductans is 0.69 uH!
Another calculation [1=Yes, 0=No] ?1
Enter diameter in mm ?12
Enter length in mm ?12
Enter number of turns ?12
Inductans is 1.19 uH!
Another calculation [1=Yes, 0=No] ?0
SLCIC Done.
```

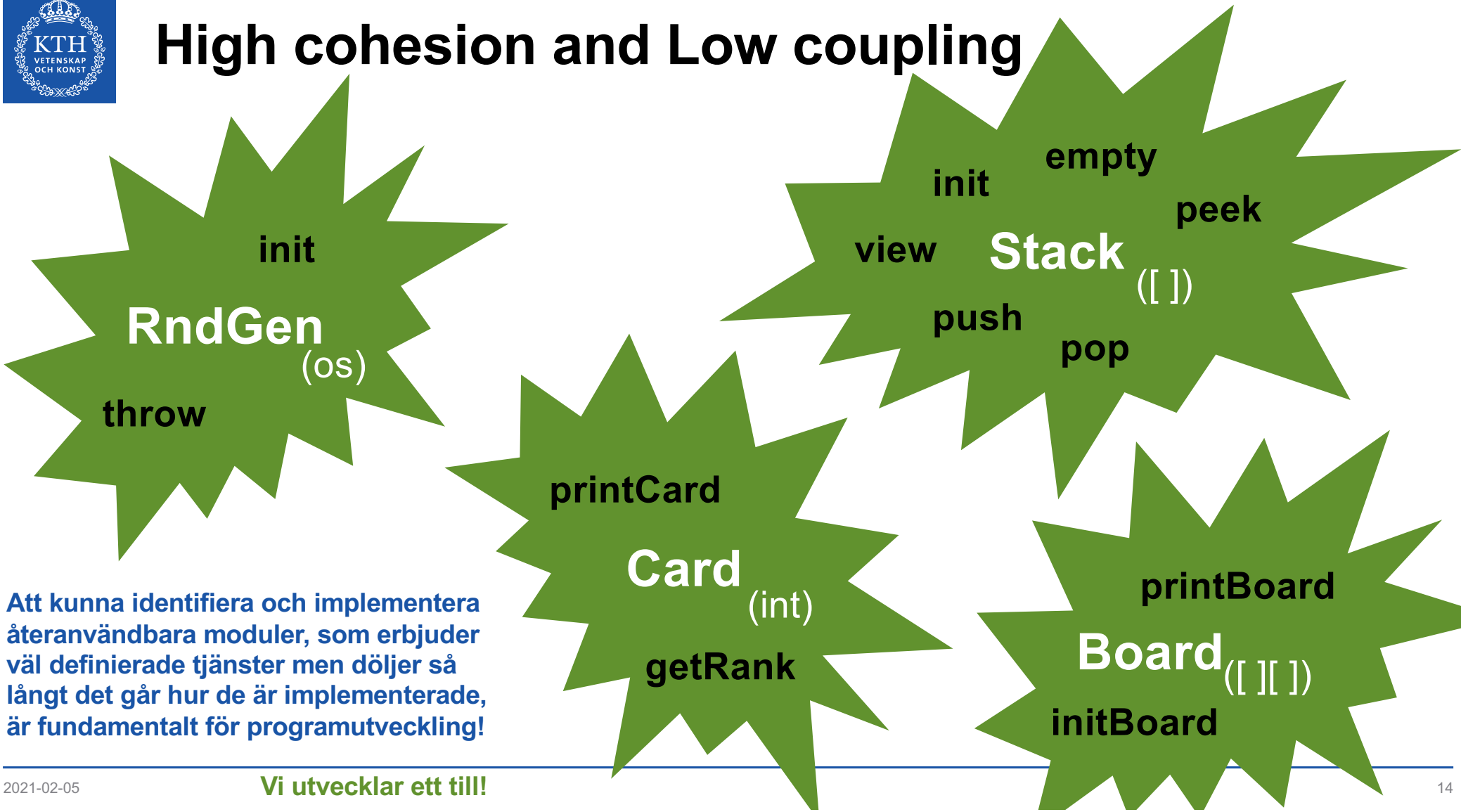
2. (4p) När Morse tänkt ut att bokstäver skulle kunna sändas som korta och långa signaler blev frågan vilken kod varje bokstav skulle få. Det sägs att han då tog en bok ur bokhyllan och noterade hur många gånger varje enskild bokstav fanns på en slumpmässigt vald sida. Han fann att bokstaven 'e' var vanligast följt av 't', 'i', 'a', 'n' och 'm'. Han gav därför de bokstäverna de kortaste koderna '.', '-', '..', '.-', '-.' & '--'. Nu är frågan: Hade Samuel Morse rätt?

För att undersöka det behövs med fördel en lång text men ditt program nöjer sig med att analysera en lång mening. Med andra ord: Uppgiften är att skapa ett program som läser in en lång mening **som ska lagras i en array**. Därefter ska programmet gå igenom arrayen och notera förekomsten av varje enskild bokstav i en frekvenstabell. Sist ska programmet skriva ut både frekvenstabellen och vilken bokstav som var vanligast. Skulle flera bokstäver få samma toppnotering räcker det om en av dessa presenteras. En körning bör se ut som följande exempel: (**Programmet ska enbart hantera 'A'..'Z'**.)

```
Text Letter Frequence Analyser.
warning: this program uses gets(), which is unsafe.
Please enter a short text: SITTING IN AN ENGLISH GARDEN WAITING FOR THE SUN
IF THE SUN DONT COME YOU GET A TAN FROM STANDING IN THE ENGLISH RAIN
 A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
 7 0 1 3 8 3 7 5 11 0 0 2 2 15 5 0 0 4 6 10 3 0 1 0 1 0
Most frequent letter is 'N' found 15 times.
TLFA - Done.
```


Nix, gör dem om, och om igen, tills de sitter!

High cohesion and Low coupling



Att kunna identifiera och implementera återanvändbara moduler, som erbjuder väl definierade tjänster men döljer så långt det går hur de är implementerade, är fundamentalt för programutveckling!

Vi utvecklar ett till!

Frekvenstabell, med nya ögon…

Skriv ett program som slumpar ett tärningskast n gånger. Programmet skall därefter skriva ut en frekvenstabell över observationerna.

```
Enter nb of atempts : 1000
 x : n
 1 : 168
 2 : 168
 3 : 160
 4 : 157
 5 : 170
 6 : 177
FTT Done, bye!
```

// Läs in antalet kast : x

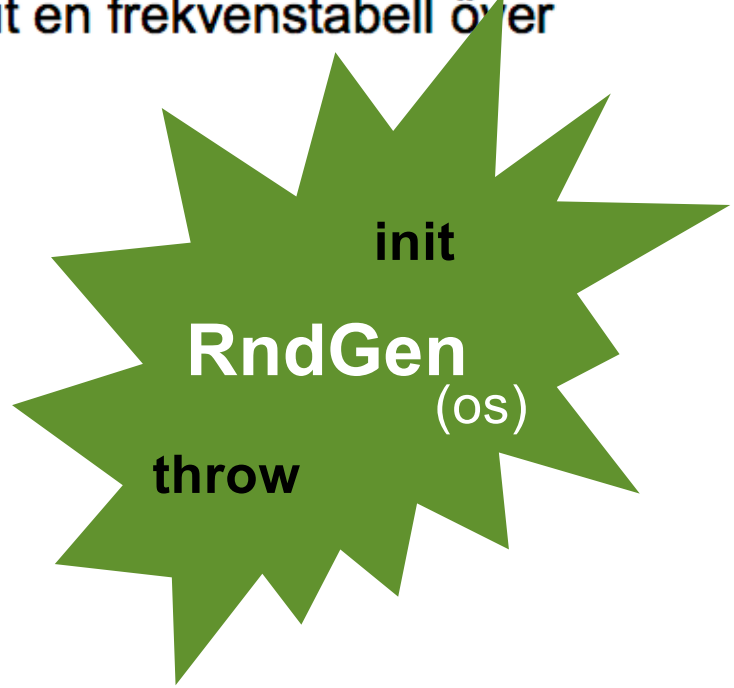
// Repetera x ggr
// Markera kast i fre.tbl.

// Visa resultatet

Frekvenstabell, med nya ögon…

Skriv ett program som slumpar ett tärningskast n gånger. Programmet skall därefter skriva ut en frekvenstabell över observationerna.

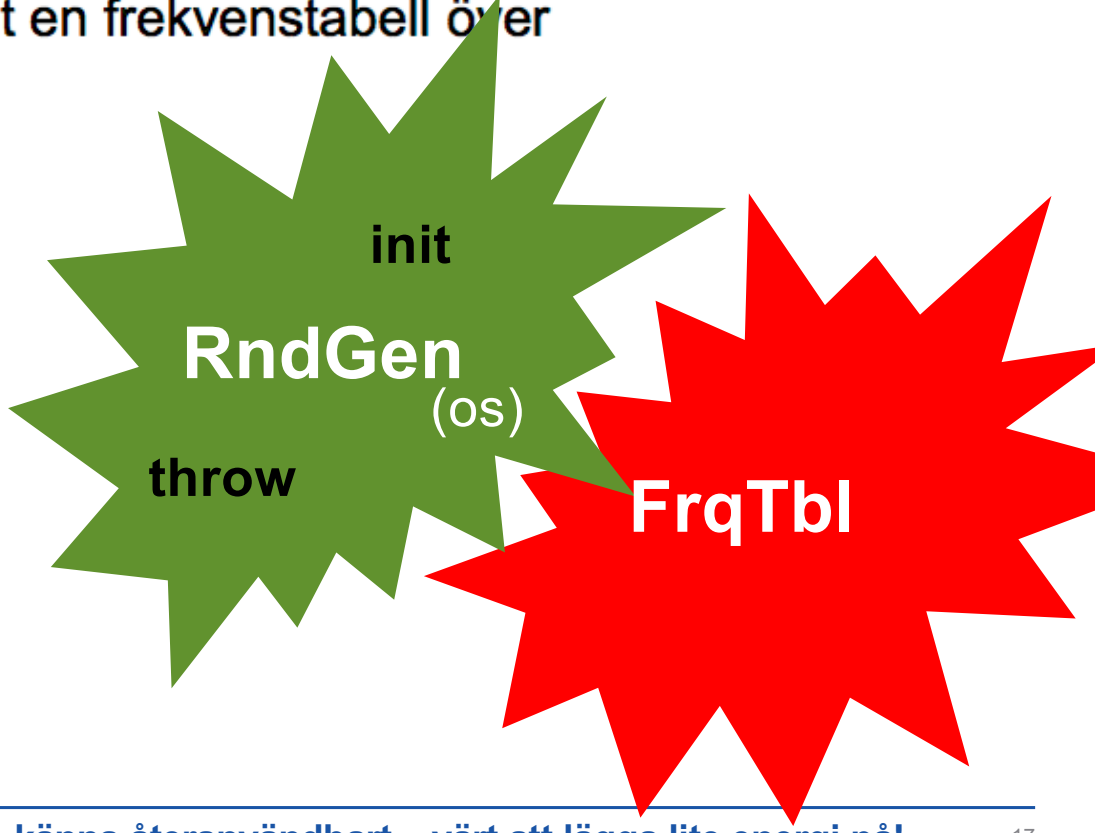
```
Enter nb of atempts : 1000
 x : n
 1 : 168
 2 : 168
 3 : 160
 4 : 157
 5 : 170
 6 : 177
FTT Done, bye!
```



Frekvenstabell, med nya ögon…

Skriv ett program som slumpar ett tärningskast n gånger. Programmet skall därefter skriva ut en frekvenstabell över observationerna.

```
Enter nb of atempts : 1000
 x : n
 1 : 168
 2 : 168
 3 : 160
 4 : 157
 5 : 170
 6 : 177
FTT Done, bye!
```

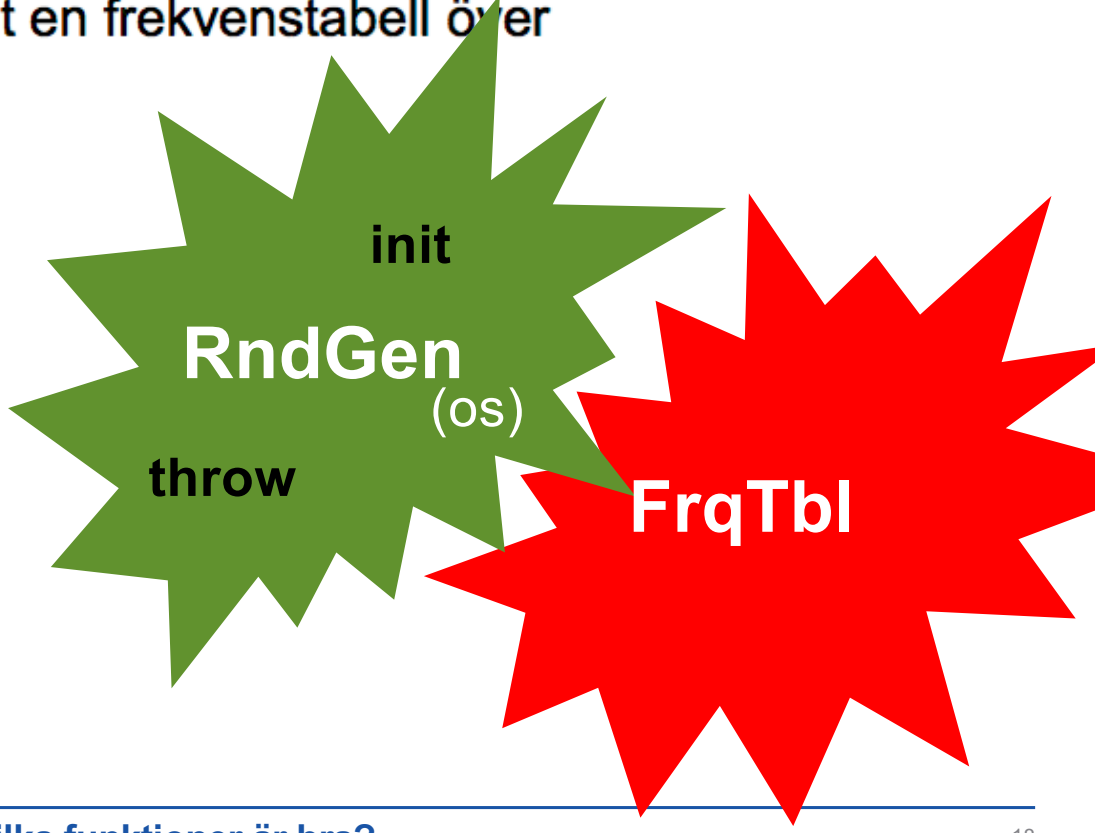


Hum, det saknas rutiner för en frekvenstabell, känns återanvändbart – värt att lägga lite energi på!

Frekvenstabell, med nya ögon…

Skriv ett program som slumpar ett tärningskast n gånger. Programmet skall därefter skriva ut en frekvenstabell över observationerna.

```
Enter nb of atempts : 1000
 x : n
 1 : 168
 2 : 168
 3 : 160
 4 : 157
 5 : 170
 6 : 177
FTT Done, bye!
```

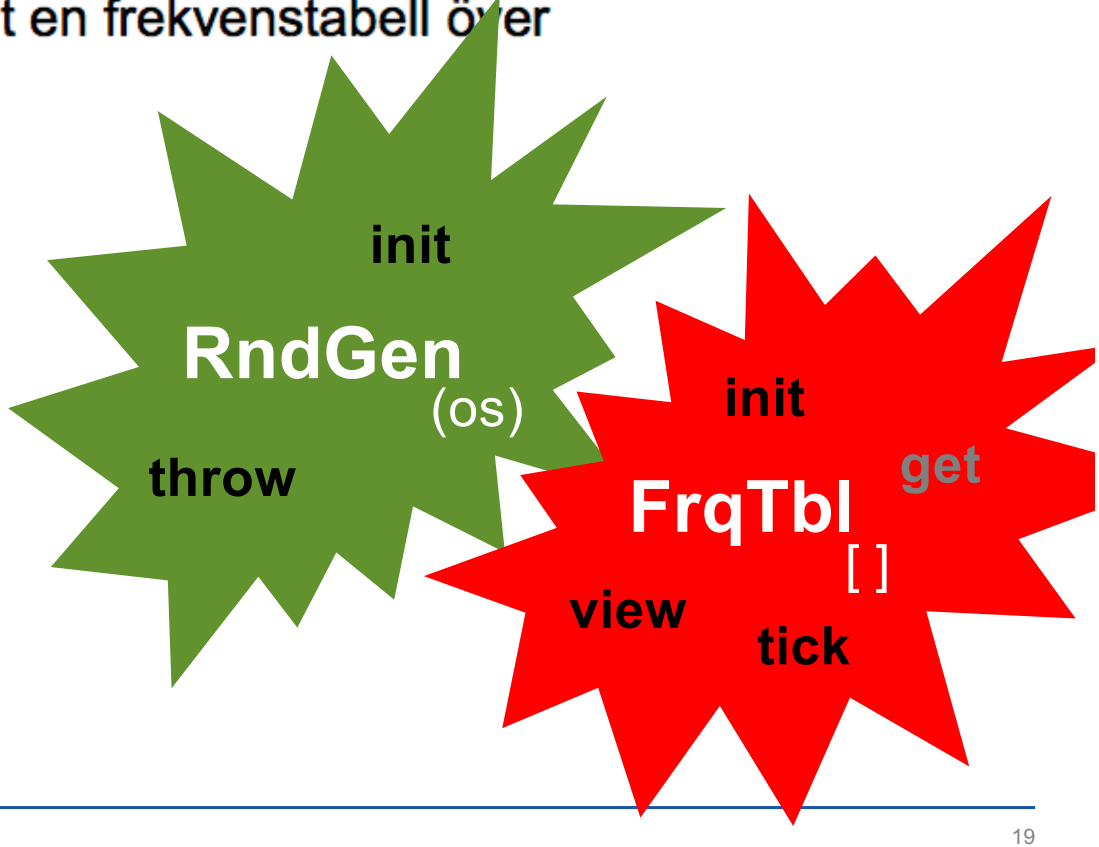


Vilken datastruktur skulle underlätta mest? Vilka funktioner är bra?

Frekvenstabell, med nya ögon…

Skriv ett program som slumpar ett tärningskast n gånger. Programmet skall därefter skriva ut en frekvenstabell över observationerna.

```
Enter nb of atempts : 1000
 x : n
 1 : 168
 2 : 168
 3 : 160
 4 : 157
 5 : 170
 6 : 177
FTT Done, bye!
```



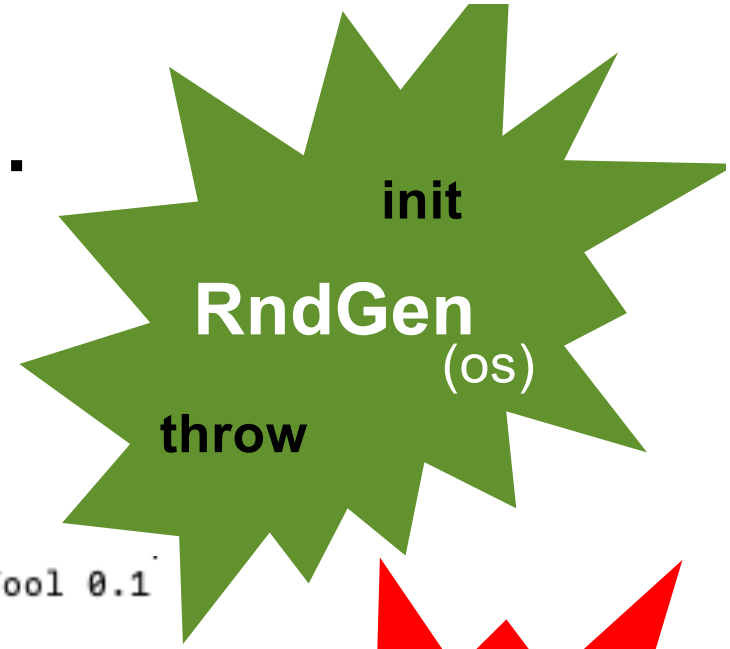
Frekvenstabell, med nya ögon…

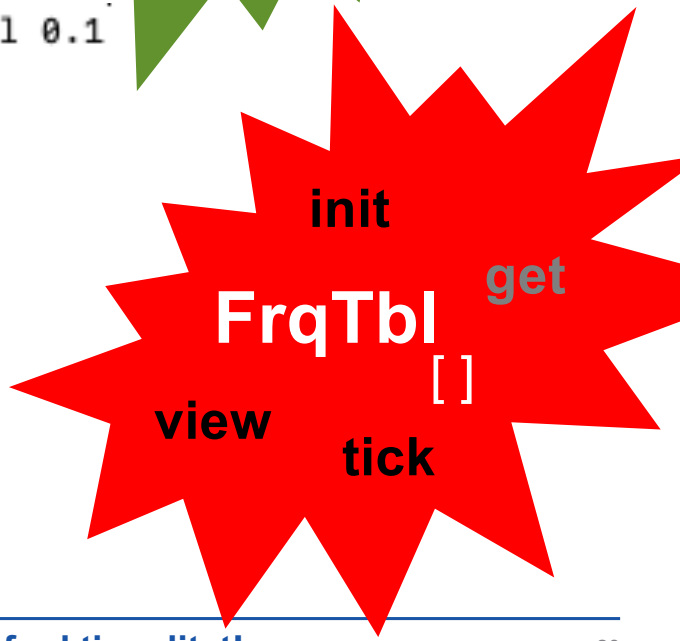
```
#include <stdio.h>

int main(int argc, char **argv)
{
    printf("Frequency Tabel Tool 0.1\n");

    printf("FTT Done, bye!\n");
    return 0;
}
```

```
Frequency Tabel Tool 0.1
FTT Done, bye!
```





Som alltid, vi gör ett skal så att vi successivt kan addera och testa ny funktionalitet!

Frekvenstabell, med nya ögon…

```
#include <stdio.h>

int m#include <stdio.h>
{   #define FTSIZE 6
    p
    r
}
    void viewFrqTbl(int qTbl[], int size) {

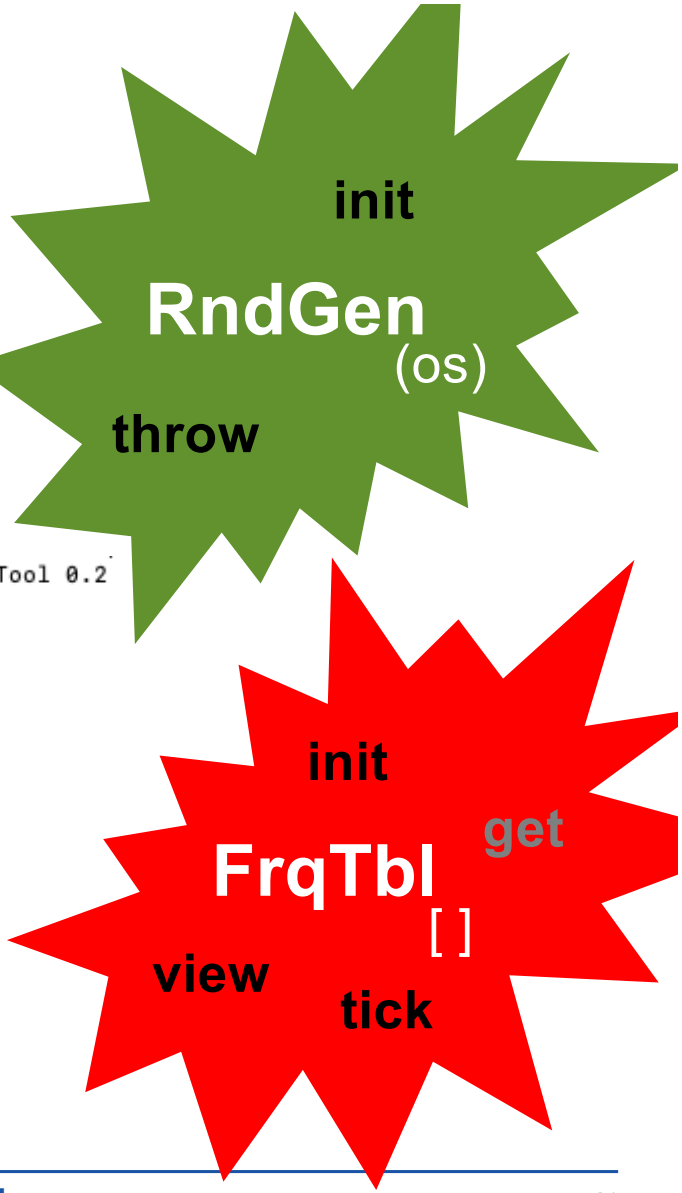

    int main(int argc, char **argv) {
        int frqTbl[FTSIZE];

        printf("Frequency Tabel Tool 0.2\n");
        initFrqTbl(
        viewFrqTbl(                    //UT: Init

        printf("FTT Done, bye!\n");
        return 0;
    }
```

”Frekvenstabellen”, men tänk på att aldrig ändra direkt i variabeln, använd ”självklara” funktioner!

```
Frequency Tabel Tool 0.2
 x : n
 1 : 0
 2 : 0
 3 : 0
 4 : 0
 5 : 0
 6 : 0
FTT Done, bye!
```



Börja alltid med datatypen, initiering och ett sätt att skriva ut objektet!

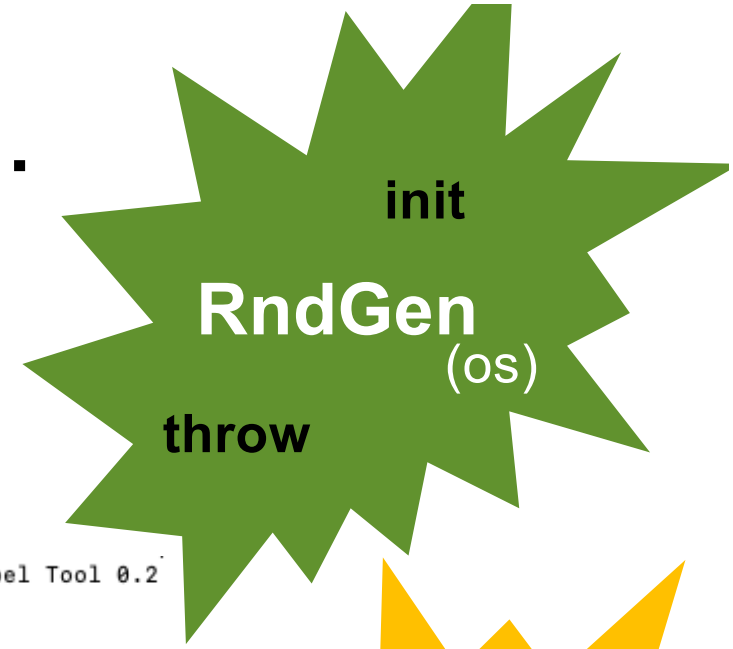
Frekvenstabell, med nya ögon…

```c
#include <stdio.h>
#define FTSIZE 6
void initFrqTbl(int aFT[], int size) {
    for (int i=0; i<size; i++)
        aFT[i]=0;
}
void viewFrqTbl(int aFT[], int size) {
    printf(" x : n \n");
    for (int i=0; i<size; i++)
        printf(" %d : %d \n",i+1,aFT[i]);
}

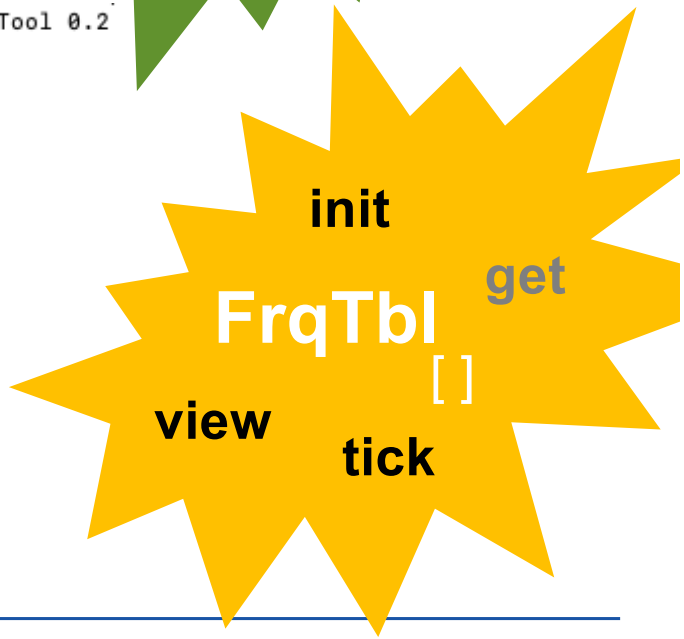
int main(int argc, char **argv) {
    int frqTbl[FTSIZE];

    printf("Frequency Tabel Tool 0.2\n");
    initFrqTbl(frqTbl, FTSIZE);
    viewFrqTbl(frqTbl, FTSIZE);  //UT: Init

    printf("FTT Done, bye!\n");
    return 0;
}
```




```
Frequency Tabel Tool 0.2
 x : n
 1 : 0
 2 : 0
 3 : 0
 4 : 0
 5 : 0
 6 : 0
FTT Done, bye!
```



Bra frqTbl är korrekt initierad, och den går att skriva ut!

Frekvenstabell, med nya ögon…

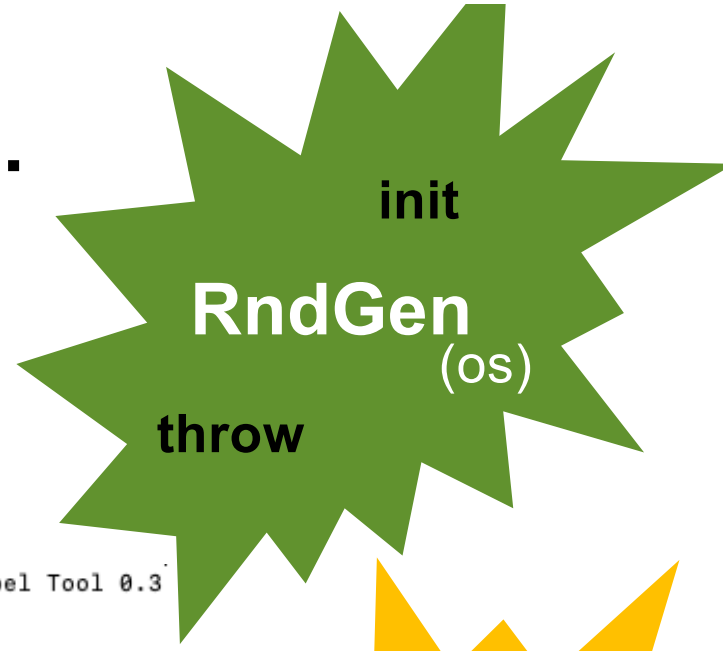
```c
#include <stdio.h>

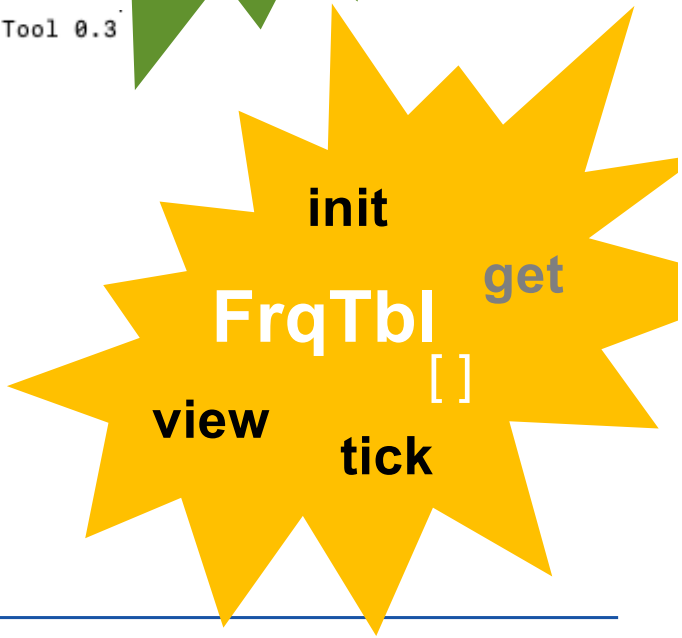
int main(int argc, char **argv) {
    int frqTbl[FTSIZE];

    printf("Frequency Tabel Tool 0.3\n");
    initFrqTbl(frqTbl, FTSIZE);
    //viewFrqTbl(frqTbl, FTSIZE);  //UT: Init
    tickFrqTbl                     //UT: Tick min
    tickFrqTbl                     //UT: Tick max
    viewFrqTbl(frqTbl, FTSIZE);  //UT: Expect 1 & 6

    printf("FTT Done, bye!\n");
    return 0;
}
```

```
Frequency Tabel Tool 0.3
 x : n
 1 : 1
 2 : 0
 3 : 0
 4 : 0
 5 : 0
 6 : 1
FTT Done, bye!
```





Nästa steg blir funktionen tickFrqTbl…!

Frekvenstabell, med nya ögon…

```c
#include <stdio.h>

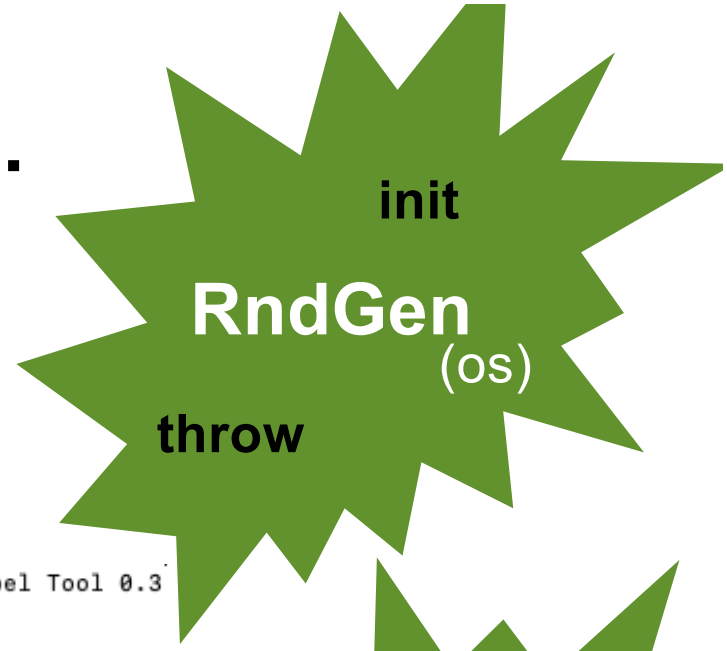
void tickFrqTbl(int aFT[], int mark) {
    aFT[mark-1]++;
}

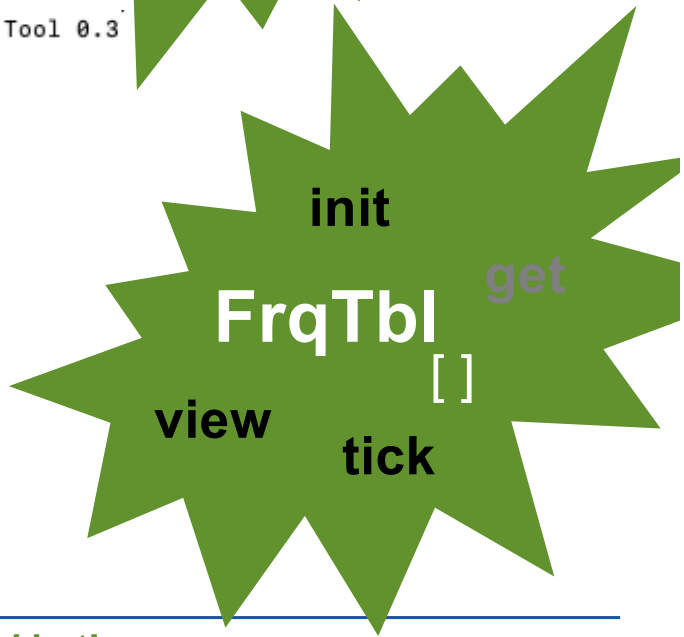
int main(int argc, char **argv) {
    int frqTbl[FTSIZE];

    printf("Frequency Tabel Tool 0.3\n");
    initFrqTbl(frqTbl, FTSIZE);
    //viewFrqTbl(frqTbl, FTSIZE);  //UT: Init
    tickFrqTbl(frqTbl,1);          //UT: Tick min
    tickFrqTbl(frqTbl,6);          //UT: Tick max
    viewFrqTbl(frqTbl, FTSIZE);    //UT: Expect 1 & 6

    printf("FTT Done, bye!\n");
    return 0;
}
```

```
Frequency Tabel Tool 0.3
 x : n
 1 : 1
 2 : 0
 3 : 0
 4 : 0
 5 : 0
 6 : 1
FTT Done, bye!
```





Ser bra ut, det är klokt att testa extremvärden! Wow, FrqTbl paketet är klart!

Frekvenstabell, med nya ögon…

```c
//*** Random Number Package *************** 1.0/AC ***
void initRandomGenerator(void){
    srand((unsigned)time(NULL));
}
int rndNumber(int nbs){
    return rand()%nbs;
}
//*** End of RNP ************************************

int main(int argc, char **argv) {
    int frqTbl[FTSIZE];

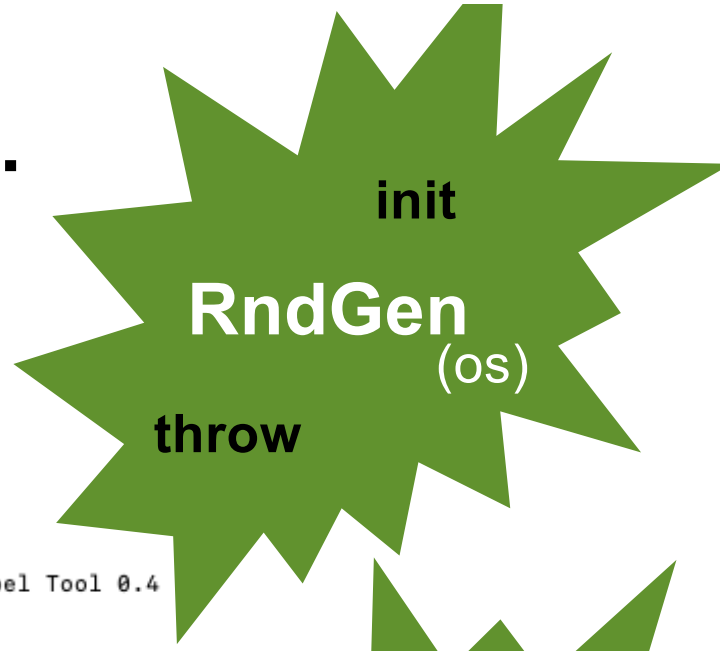
    printf("Frequency Tabel Tool 0.4\n");
    initRandomGenerator();
    initFrqTbl(frqTbl, FTSIZE);
    //viewFrqTbl(frqTbl, FTSIZE);  //UT: Init
    //tickFrqTbl(frqTbl,1);        //UT: Tick min
    //tickFrqTbl(frqTbl,6);        //UT: Tick max
    //viewFrqTbl(frqTbl, FTSIZE);  //UT: Expect 1 & 6
    tickFrqTbl(frqTbl,
               rndNumber(FTSIZE)+1);//UT:rndNumber
    viewFrqTbl(frqTbl, FTSIZE);     //UT: Expect 1..6

    printf("FTT Done, bye!\n");
    return 0;
}
```

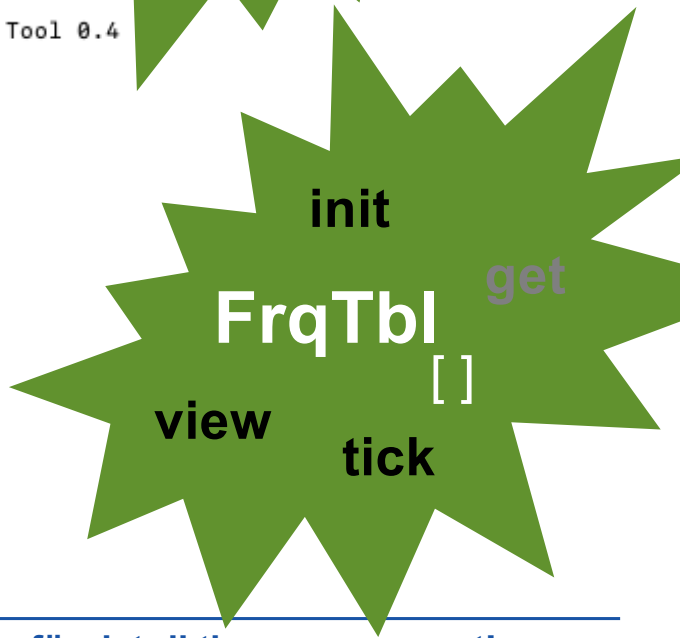
```c
rk) {

0.3\n");

  //UT: Init
//UT: Tick min
//UT: Tick max
//UT: Expect 1 & 6
```




```
Frequency Tabel Tool 0.4
 x : n
 1 : 0
 2 : 1
 3 : 0
 4 : 0
 5 : 0
 6 : 0
FTT Done, bye!
```



Kör det några ggr,
det borde bli en 1:a
på lite olika ställen!

Då är det dags att lyfta in slumprutinerna… …verkar fungera bra… dag för det riktiga programmet!

Frekvenstabell, med nya ögon…

```
int main(int argc, char **argv) {
    int frqTbl[FTSIZE];
    int nb=36;

    printf("Frequency Tabel Tool 0.5\n");
    initRandomGenerator();
    initFrqTbl(frqTbl, FTSIZE);

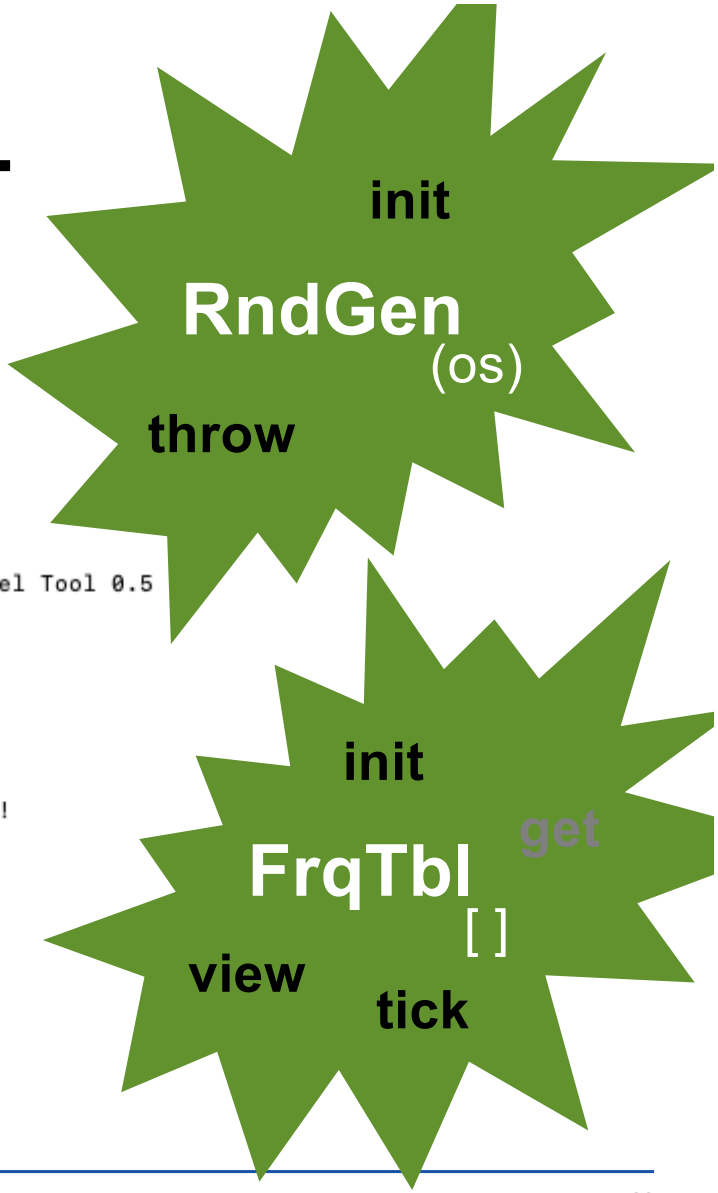
    // Fråga efter hur många kast

    for (int i=0; i<nb; i++)
        tickFrqTbl(frqTbl,rndNumber(FTSIZE)+1);

    viewFrqTbl(frqTbl, FTSIZE);

    printf("FTT Done, bye!\n");
    return 0;
}
```

```
Frequency Tabel Tool 0.5
 x : n
 1 : 4
 2 : 6
 3 : 5
 4 : 9
 5 : 6
 6 : 6
FTT Done, bye!
```



Yes… då återstår enbart den riktiga användardialogen…!

Frekvenstabell, med nya ögon…

```
int main(int argc, char **argv) {
    int frqTbl[FTSIZE];
    int nb=36;

    printf("Frequency Tabel Tool 1.0\n");
    initRandomGenerator();
    initFrqTbl(frqTbl, FTSIZE);

    printf("Enter nb of atempts : ");
    scanf("%d", &nb);

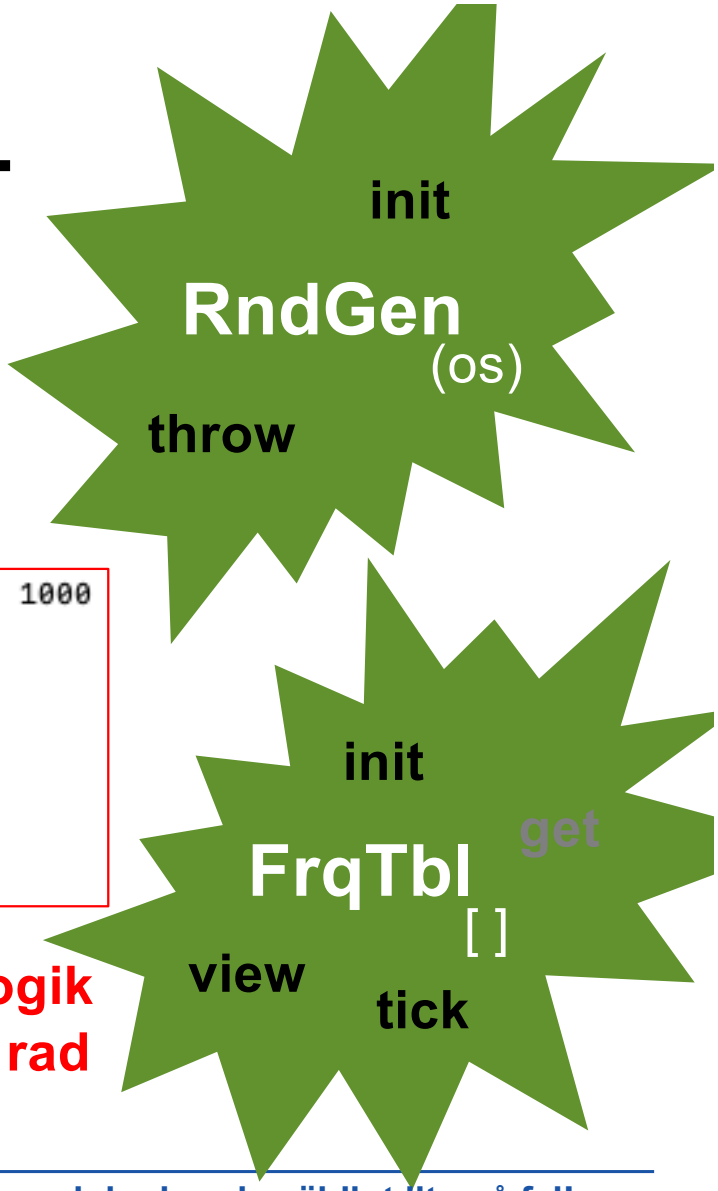
    for (int i=0; i<nb; i++)
        tickFrqTbl(frqTbl,rndNumber(FTSIZE)+1);

    viewFrqTbl(frqTbl, FTSIZE);

    printf("FTT Done, bye!\n");
    return 0;
}
```

```
Enter nb of atempts : 1000
 x : n
 1 : 168
 2 : 168
 3 : 160
 4 : 157
 5 : 170
 6 : 177
FTT Done, bye!
```

Notera att all logik finns på en (1) rad



Tack vare att huvudprogrammet byggde på separatutvecklade och testade moduler kunde väldigt lite gå fel!

Nytt: Pekare

```
/* Parallell
#include <st
int main(voi
    int r1;
    int r2;
    int rx;

    printf("P

    printf("First resistor  (ohm): ");
    scanf("%d",&r1);
    printf("Second resistor (ohm): ");
    scanf("%d",&r2);
    rx=(r1*r2)/(r1+r2);
    printf("Replace with a %d ohm resistor.\n",rx);

    printf("PRC Done.\n");
    return 0;
}
```

%d = ”Läs in ett heltal!”

```
scanf("%d",&r1);
```

Det här är en sats.

Till variabeln som finns på den här platsen! &r1=vart=8_{10}, r1=innehållet=4

(& = Adressoperator.)




```
vcxxs-MacBook:~ vcxx$ /Users/vcxx/Desktop/main
Parallel Resistor Calcylator (PRC)
First resistor  (ohm): 4
Second resistor (ohm): 6
Replace with a 2 ohm resisto
PRC Done.
vcxxs-MacBook:~ vcxx$
```

”Programmet” **stannar upp** här och väntar på att **du** matar in ett tal + retur!

OK!

Hum… …scanf() kan uppenbart ta emot, och använda, adressen till en int?!

Nytt: Pekare

```c
// Parallell resistor 2.0
#include <stdio.h>

int main(void){
    int r1 = 4;
    int r2 = 6;
    int rx;

    printf("Parallell Resistor Calcylator 2.0\n");

    printf("Value of r1 = %d\n", r1);
    printf("In memmory cell %p\n", &r1);

    printf("Parallell Resistor Calcylator Done\n");
}
```

```
 PROGRAM
#0000: CODE
#0001: CODE
#0010: CODE
   *
   *
 VART              VÄRDE  NAMN  TYP
#7FFF 55F1 AC1C:    4      r1   int
#7FFF 55F1 AC1D:    6      r2   int
#7FFF 55F1 AC1E:    ?      rx   int
#7FFF 55F1 AC1F:
#7FFF 55F1 AC20:
```

```
Parallell Resistor Calcylator 2.0
Value of r1 = 4
In memmory cell 0x7fff55f1ac1c
Parallell Resistor Calcylator Done
```

%p skriver ut en adress, som tydligen kan vara ett mycket stort tal…

Hum, &r1 ger alltså adressen till den minnes-cell som kompilatorn har valt att spara r1:s värde i (tur att vi slipper komma ihåg den!)…

Hum… …går det att spara den adressen i en variabel, det är trots allt ett tal?!

Nytt: Pekare

```c
// Parallell resistor 2.0
#include <stdio.h>

int main(void){
    int r1 = 4;
    int r2 = 6;
    int rx;
    int *pR1 = &r1;

    printf("Parallell Resistor Calcylator 2.0\n");

    printf("Value of r1 = %d\n", r1);
    printf("In memmory cell %p\n", pR1);

    printf("Parallell Resistor Calcylator Done\n");
}
```

```
  PROGRAM
#0000: CODE
#0001: CODE
#0010: CODE
    *
    *
 VART              VÄRDE  NAMN  TYP
#7FFF 55F1 AC1C:    4      r1    int
#7FFF 55F1 AC1D:    6      r2    int
#7FFF 55F1 AC1E:    ?      rx    int
#7FFF 55F1 AC1F:  7…1C     pR1   int*
#7FFF 55F1 AC20:
```

```
Parallell Resistor Calcylator 2.0
Value of r1 = 4
In memmory cell 0x7fff55f1ac1c
Parallell Resistor Calcylator Done
```

Det är väldigt smart (krav) att pekarvariabler får prefixet 'p' i sitt namn (alternativt post fix Pointer eller Ptr)!

Genom att skapa en variabel av typen int * (utläses som adressen till en int) kommer kompilatorn ihåg:

1) innehållet är en adress
2) till en minnescell som **ska** innehålla en int!

Hum… …då borde det gå att använda pR1 i stället för &r1 som argument?!

Nytt: Pekare

```c
// Parallell resistor 2.0
#include <stdio.h>

int main(void){
    int r1 = 4;
    int r2 = 6;
    int rx;
    int *pR1 = &r1;

    printf("Parallell Resistor Calcylator 2.0\n");

    printf("Value of r1 = %d\n", r1);

    printf("Enter value of r1 ? ");
    scanf("%d", pR1);

    printf("Value of r1 = %d\n", r1);

    printf("Parallell Resistor Calcylator Done\n");
}
```

```
 PROGRAM
#0000: CODE
#0001: CODE
#0010: CODE
   *
   *
 VART                VÄRDE  NAMN  TYP
#7FFF 55F1 AC1C:     4 8     r1    int
#7FFF 55F1 AC1D:     6       r2    int
#7FFF 55F1 AC1E:     ?       rx    int
#7FFF 55F1 AC1F:   7…1C      pR1   int*
#7FFF 55F1 AC20:
```

```
Parallell Resistor Calcylator 2.0
Value of r1 = 4
Enter value of r1 ? 8
Value of r1 = 8
Parallell Resistor Calcylator Done
```

Obs: Deklar. av pR1 ger enbart plats för att lagra en adress, det minnesutrymme som adressen anger **måste också skapas**!

Hum… …kan programmet göra det som scanf() kan, om det har en adress?!

Nytt: Pekare

```
#include <stdio.h>

int main(void){
    char ch = 'A';
    char *pCh = &ch;  // & Ge mig adressen till…

    printf("Pointer tester 1.0\n");

    printf("Value of ch = %c\n", ch);

    *pCh = 'B';       // * Innehållet i adressen…

    printf("Value of ch = %c\n", ch);

    printf("Pointer tester 1.0 Done\n");
}
```

```
 PROGRAM
#0000: CODE
#0001: CODE
#0010: CODE
   *
   *
 VART              VÄRDE NAMN TYP
#7FFF 55F1 AC1C:~~'A'~~'B' ch   char
#7FFF 55F1 AC1D:7…1C       pCh  char*
#7FFF 55F1 AC1E:
#7FFF 55F1 AC1F:
#7FFF 55F1 AC20:
```

```
Pointer tester 1.0
Value of ch = A
Value of ch = B
Pointer tester 1.0 Done
```

Wow: & ger adressen till en variabel, och * innehållet i en adress

Hum, vad händer om vi använder en adress som ett funktionsargument?

Nytt: Pekare

```c
#include <stdio.h>

void times2(int number) {
   printf("...double in  :%d\n",number);
   number=number*2;
   printf("...double out :%d\n",number);
}

int main(void){
  int nb=7;

  printf("Functions & pointers!\n");
  printf("Before call   :%d\n",nb);
  times2(nb);
  printf("After call    :%d\n",nb);

  return 0;
}
```



```
  PROGRAM
#0000: CODE
#0001: CODE
#0010: CODE
    *
    *
 VART    VÄRDE       NAMN       TYP
#7…1C:     7          nb        int
#7…1D:
#7…1E:
#7…1F:
#7…20:
```

```
Functions & pointers!
Before call   :7
...double in  :7
...double out :14
After call    :7
```


Först, repetition, utan adress… …ser ok ut, en lokal variabel!

Nytt: Pekare

```c
#include <stdio.h>

void times2(int number) {
   printf("...double in  :%d\n",number);
   number=number*2;
   printf("...double out :%d\n",number);
}

int main(void){
  int nb=7;

  printf("Functions & pointers!\n");
  printf("Before call   :%d\n",nb);
  times2(nb);
  printf("After call    :%d\n",nb);

  return 0;
}
```



```
  PROGRAM
#0000: CODE
#0001: CODE
#0010: CODE
    *
    *
 VART    VÄRDE      NAMN      TYP
#7…1C:     7         nb       int
#7…1D:     7        number    int
#7…1E:
#7…1F:
#7…20:
```

```
Functions & pointers!
Before call   :7
...double in  :7
...double out :14
After call    :7
```


Först, repetition, utan adress… …ser ok ut, en lokal variabel!

Nytt: Pekare

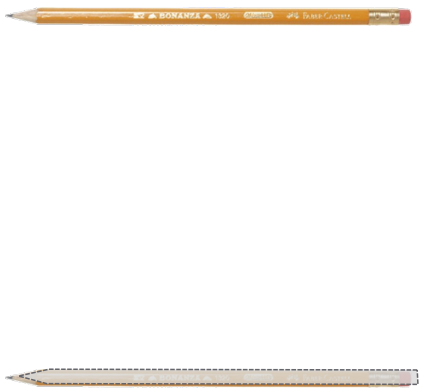
```c
#include <stdio.h>

void times2(int number) {
   printf("...double in  :%d\n",number);
   number=number*2;
   printf("...double out :%d\n",number);
}

int main(void){
  int nb=7;

  printf("Functions & pointers!\n");
  printf("Before call   :%d\n",nb);
  times2(nb);
  printf("After call    :%d\n",nb);

  return 0;
}
```



```
  PROGRAM
#0000: CODE
#0001: CODE
#0010: CODE
    *
    *
 VART    VÄRDE      NAMN      TYP
#7…1C:     7          nb       int
#7…1D:     7 14       number   int
#7…1E:
#7…1F:
#7…20:
```

```
Functions & pointers!
Before call   :7
...double in  :7
...double out :14
After call    :7
```


Yes, den lokala (parameter-)variabeln dubbleras!

Nytt: Pekare

```c
#include <stdio.h>

void times2(int number) {
   printf("...double in  :%d\n",number);
   number=number*2;
   printf("...double out :%d\n",number);
}

int main(void){
  int nb=7;

  printf("Functions & pointers!\n");
  printf("Before call   :%d\n",nb);
  times2(nb);
  printf("After call    :%d\n",nb);

  return 0;
}
```



```
  PROGRAM
#0000: CODE
#0001: CODE
#0010: CODE
    *
    *
 VART    VÄRDE       NAMN       TYP
#7…1C:     7          nb        int
#7…1D:
#7…1E:
#7…1F:
#7…20:

Functions & pointers!
Before call   :7
...double in  :7
...double out :14
After call    :7
```


Men det påverkar inte argumentet, som har kvar sitt värde (copy-by-value)!

Nytt: Pekare

(Funktionen tar emot ett heltal)

```c
#include <stdio.h>

void times2(int number) {
   printf("...double in  :%d\n",number);
   number=number*2;
   printf("...double out :%d\n",number);
}

int main(void){
  int nb=7;

  printf("Functions & pointers!\n");
  printf("Before call   :%d\n",nb);
  times2(nb);
  printf("After call    :%d\n",nb);

  return 0;
}
```

(Vi skickar en kopia av nb:s värde)

(Funktionen tar emot en adress)

```c
#include <stdio.h>

void times2(int *nbPtr) {
   printf("...double in  :%d\n",*nbPtr);
   *nbPtr=*nbPtr*2;
   printf("...double out :%d\n",*nbPtr);
}

int main(void){
  int nb=7;

  printf("Functions & pointers!\n");
  printf("Before call   :%d\n",nb);
  times2(&nb);
  printf("After call    :%d\n",nb);

  return 0;
}
```

(Vi skickar vart nb finns i minnet)

Vad händer nu om vi skickar adressen till vart argumentet finns i stället?

Nytt: Pekare

```c
#include <stdio.h>

void times2(int *nbPtr) {
   printf("...double in  :%d\n",*nbPtr);
   *nbPtr=*nbPtr*2;
   printf("...double out :%d\n",*nbPtr);
}

int main(void){
  int nb=7;

  printf("Functions & pointers!\n");
  printf("Before call   :%d\n",nb);
  times2(&nb);
  printf("After call    :%d\n",nb);

  return 0;
}
```



```
 PROGRAM
#0000: CODE
#0001: CODE
#0010: CODE
    *
    *
 VART    VÄRDE       NAMN       TYP
#7…1C:     7          nb        int
#7…1D:
#7…1E:
#7…1F:
#7…20:
```

```
Functions & pointers!
Before call   :7
...double in  :7
...double out :14
After call    :14
```

Nytt: Pekare

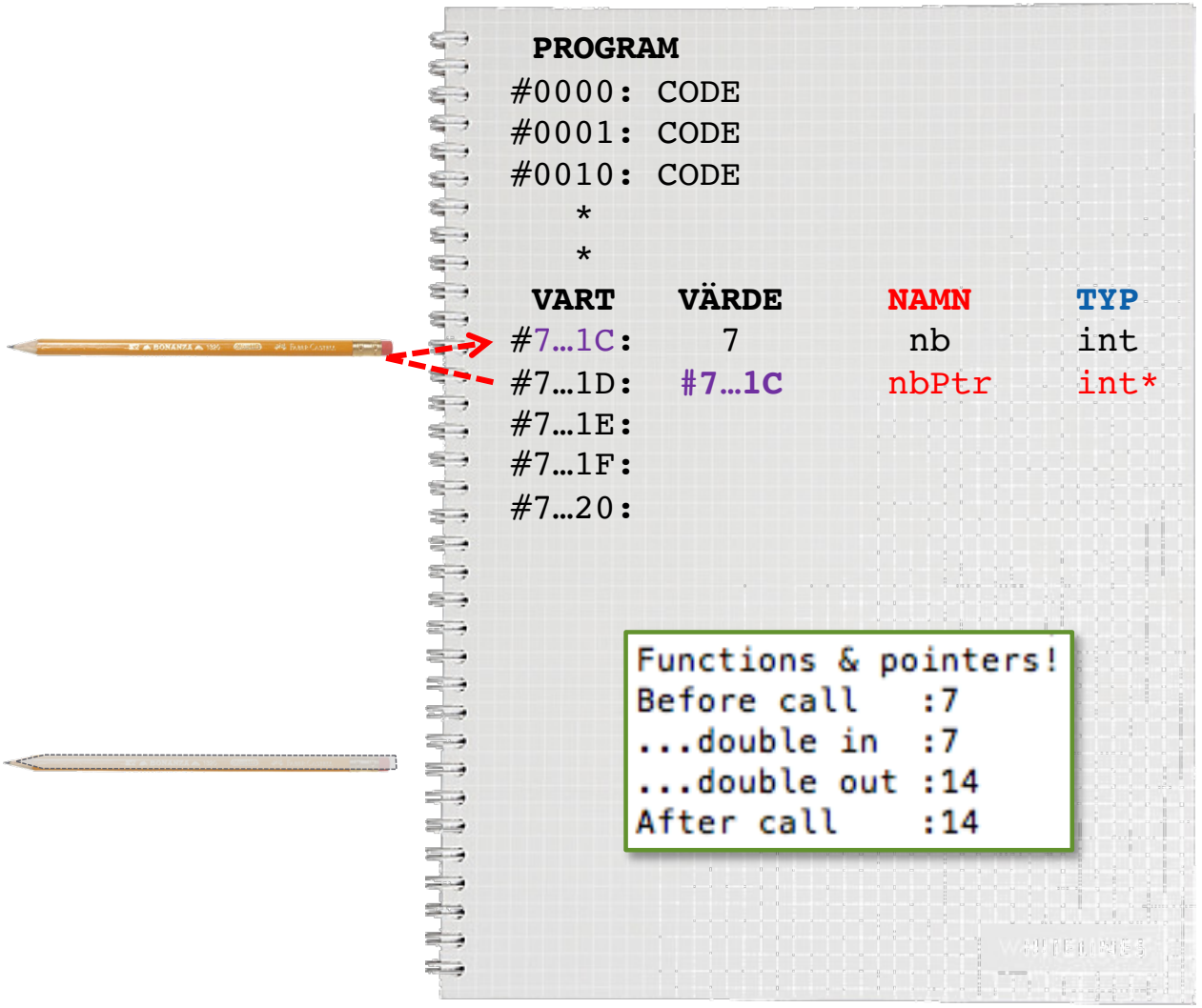
```c
#include <stdio.h>

void times2(int *nbPtr) {
   printf("...double in  :%d\n",*nbPtr);
   *nbPtr=*nbPtr*2;
   printf("...double out :%d\n",*nbPtr);
}

int main(void){
  int nb=7;

  printf("Functions & pointers!\n");
  printf("Before call   :%d\n",nb);
  times2(&nb);
  printf("After call    :%d\n",nb);

  return 0;
}
```



Rätt lika, men parametervariabeln innehåller nu adressen till vart nb finns, inte nb:s värde…

Nytt: Pekare

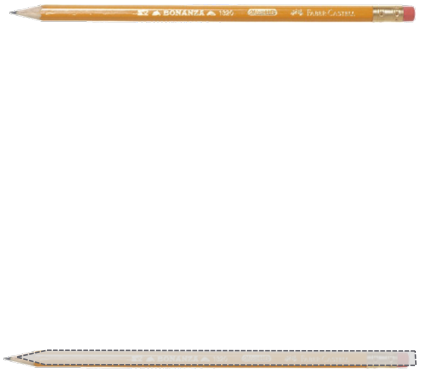
```c
#include <stdio.h>

void times2(int *nbPtr) {
   printf("...double in  :%d\n",*nbPtr);
   *nbPtr=*nbPtr*2;
   printf("...double out :%d\n",*nbPtr);
}

int main(void){
  int nb=7;

  printf("Functions & pointers!\n");
  printf("Before call   :%d\n",nb);
  times2(&nb);
  printf("After call    :%d\n",nb);

  return 0;
}
```



```
  PROGRAM
#0000: CODE
#0001: CODE
#0010: CODE
    *
    *
 VART    VÄRDE      NAMN      TYP
#7…1C:    7 14       nb       int
#7…1D:   #7…1C     nbPtr      int*
#7…1E:
#7…1F:
#7…20:
```

```
Functions & pointers!
Before call   :7
...double in  :7
...double out :14
After call    :14
```


…Wow, raden dubblerar innehållet i den utpekade minnescellen, som är anropande funktions argument!

Nytt: Pekare

```c
#include <stdio.h>

void times2(int *nbPtr) {
   printf("...double in  :%d\n",*nbPtr);
   *nbPtr=*nbPtr*2;
   printf("...double out :%d\n",*nbPtr);
}

int main(void){
  int nb=7;

  printf("Functions & pointers!\n");
  printf("Before call   :%d\n",nb);
  times2(&nb);
  printf("After call    :%d\n",nb);

  return 0;
}
```



```
  PROGRAM
#0000: CODE
#0001: CODE
#0010: CODE
    *
    *
 VART    VÄRDE       NAMN       TYP
#7…1C:    7 14        nb        int
#7…1D:
#7…1E:
#7…1F:
#7…20:

Functions & pointers!
Before call   :7
...double in  :7
...double out :14
After call    :14
```


…åter i den anropande funktionens kod så har argumentet värde ändrats magiskt!



Sammantaget

1. En ej initierad pekarvariabel är ett slumptal, som kan avsluta programmet om pekaren används (Segmentation Fault)!
2. Makrot **NULL** kan användas för att ge en pekare ett värde som betyder att pekaren för stunden inte pekar på något!
3. Flera pekarvariabler kan peka på samma variabel!
4. Glöm aldrig & eller * där de krävs, det blir alltid svåra fel!
5. **Variabelnamnet på en matris (utan [ ]) kan användas som en pekare till första elementet, det är därför & inte behövs i funktionsanrop**.
6. Döp alltid pekarvariabler med prefixet p (eller möjligen postfix Pointer alt Ptr).
7. Innebörden av *&r1 är r1, d.v.s r1:s värde, ty tolkas *(&r1)!

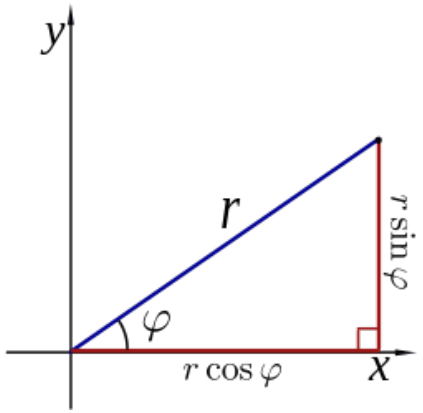


Varför lär vi oss pekare?

1. **Pekare möjliggör att funktioner kan ändra värdet på anropande funktionsargument. Helt avgörande kunskap för att klara denna kurs! (Utan denna egenskap kan en funktion bara returnera ett värde.)**
2. **Eftersom c kopierar funktionsargument blir funktionsanropet mycket snabbare om en pekare till argumentet används! Vi kommer att se det senare i denna kurs!**
3. Pekare möjliggör att ett program kan skapa variabler när de behövs dynamiskt, inte enbart statiskt via deklarationer. Helt avgörande i de flesta reella programutvecklingsprojekt, får dock vänta till senare kurser.
4. Pekare möjliggör väldigt användbara speciella dynamiska datastrukturer som kallas länkade listor & träd. Helt avgörande i de flesta reella programutvecklingsprojekt, får dock vänta till senare kurser.

Övning: Pekarparametrar!

- Skapa en funktion toCartesian som tar en polär koordinat (r,fi) och returnerar motsvarande rätvinkliga koordinater (x,y).
- (x=r*cos(fi) & y=r*sin(fi), fi anges i grader som ska konverteras till rad för att cos & sin ska fungera. Konstanten M_PI samt cos och sin finns via biblioteket math.h)
- Alla koordinater ska ha dubbel precision…




```c
int main(void){
    double x, y, r=5.0, fi=36.869897;

    printf("Pointers (PTR)!\n");

    toCartesian(r, fi, &x, &y);
    printf("[ %3.1f, %3.1f ]\n", x, y);

    printf("PTR Done.\n");
    return 0;
}
```

```
Pointers (PTR)!
[ 4.0, 3.0 ]
PTR Done.
```

Övning: Pekarparametrar!

```c
/* PTR 0.2 AC */
#include <stdio.h>
#include <math.h>

void toCartesian(double r, double fi, double *xPtr, double *yPtr);

int main(void){
    double x, y, r=5.0, fi=36.869897;

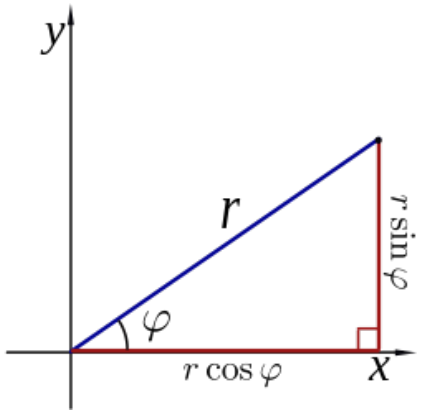
    printf("Pointers (PTR)!\n");

    toCartesian(r, fi, &x, &y);
    printf("[ %3.1f, %3.1f ]\n", x, y);

    printf("PTR Done.\n");
    return 0;
}

void toCartesian(double r, double fi, double *xPtr, double *yPtr){


}
```




```
Pointers (PTR)!
[ 4.0, 3.0 ]
PTR Done.
```

Övning: Pekarparametrar!

```c
/* PTR 0.2 AC */
#include <stdio.h>
#include <math.h>

void toCartesian(double r, double fi, double *xPtr, double *yPtr);

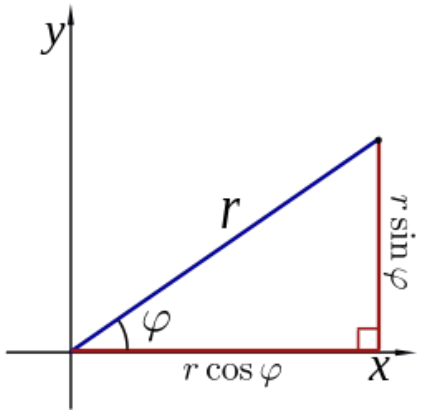
int main(void){
    double x, y, r=5.0, fi=36.869897;

    printf("Pointers (PTR)!\n");

    toCartesian(r, fi, &x, &y);
    printf("[ %3.1f, %3.1f ]\n", x, y);

    printf("PTR Done.\n");
    return 0;
}

void toCartesian(double r, double fi, double *xPtr, double *yPtr){
    *xPtr=r*cos(fi/180.0*M_PI);
    *yPtr=r*sin(fi/180.0*M_PI);
}
```




```
Pointers (PTR)!
[ 4.0, 3.0 ]
PTR Done.
```

printf() – din bästa vän!

```c
#include <stdio.h>
#define STACKSIZE 10
#define DBGLEVEL 2                          // 2..0 Most verbose to none

//*** Debug package ********************************************
void dbgEnter(char *msg) {
    switch (DBGLEVEL) {
        case 2: printf(">>> %s\n",msg); //break intensional omitted
    }
}
void dbgExit(char *msg) {
    switch (DBGLEVEL) {
        case 2: printf("<<< %s\n",msg); //break intensional omitted
    }
}
void dbgInspect(char *msg, char *id, int value) {
    switch (DBGLEVEL) {
        case 1:                             //break intensional omitted
        case 2: printf("--- %s: %s=%d\n",
                       msg, id, value); //break intensional omitted
    }
}
//*** DBGP END *************************************************
void initStack(int aStack[]){
    dbgEnter("initStack");
    aStack[0]=0;
    dbgExit("initStack");
}
void printStack(int aStack[]){
    dbgEnter("printStack");
    dbgInspect("printStack", "items", aStack[0]);
```

```
RPN Calculator 1.0
>>> initStack
<<< initStack
3 2 1 + + =
>>> printStack
--- printStack: items=1
[ 3 ]
<<< printStack
>>> printStack
--- printStack: items=2
[ 2 3 ]
<<< printStack
>>> printStack
--- printStack: items=3
[ 1 2 3 ]
<<< printStack
>>> printStack
--- printStack: items=2
[ 3 3 ]
<<< printStack
>>> printStack
--- printStack: items=1
[ 6 ]
<<< printStack
>>> printStack
--- printStack: items=0
[ ]
<<< printStack
RPNC Done, bye!
```

```
RPN Calculator 1.0
3 2 1 + + =
--- printStack: items=1
[ 3 ]
--- printStack: items=2
[ 2 3 ]
--- printStack: items=3
[ 1 2 3 ]
--- printStack: items=2
[ 3 3 ]
--- printStack: items=1
[ 6 ]
--- printStack: items=0
[ ]
RPNC Done, bye!
```

```
RPN Calculator 1.0
3 2 1 + + =
[ 3 ]
[ 2 3 ]
[ 1 2 3 ]
[ 3 3 ]
[ 6 ]
[ ]
RPNC Done, bye!
```

2 1 0

DBGLEVEL



Lab #2: Matriser & Funktioner

Att använda datorprogram för mätdatainsamling och analys är mycket vanligt. Ni ska skriva ett välstrukturerat program där man kan mata in mätvärden och analysera dessa. När programmet startar får användaren upp en meny där denne kan välja:

v (View) - visar vilka mätvärden som finns lagrade
e (Enter) - tillåter användaren att lägga till mätvärden
c (Compute) - presenterar max, min, medelvärde och normaliserade mätvärden
r (Reset) - raderar alla inmatade mätvärden
q (Quit) - avslutar programmet

Användaren ska göra sitt val genom att ange en bokstav…

Hela uppgiften finns på Canvas!

```
Measurement tool 1.2
VECRQ? r
VECRQ? v
[ No measurements ]
VECRQ? e
Enter measurement #1 (or 0) ?1
Enter measurement #2 (or 0) ?2
Enter measurement #3 (or 0) ?3
Enter measurement #4 (or 0) ?0
VECRQ? v
[ 1 2 3 ]
VECRQ? e
Enter measurement #4 (or 0) ?4
Enter measurement #5 (or 0) ?5
Enter measurement #6 (or 0) ?0
VECRQ? v
[ 1 2 3 4 5 ]
VECRQ? c
Max value : 5
Min value : 1
Avr value : 3.00
[ -2 -1 0 1 2 ]
VECRQ? q
Exit Measurement tool! _
```


Börjar vi skriva kod direkt, eller vad gör vi…

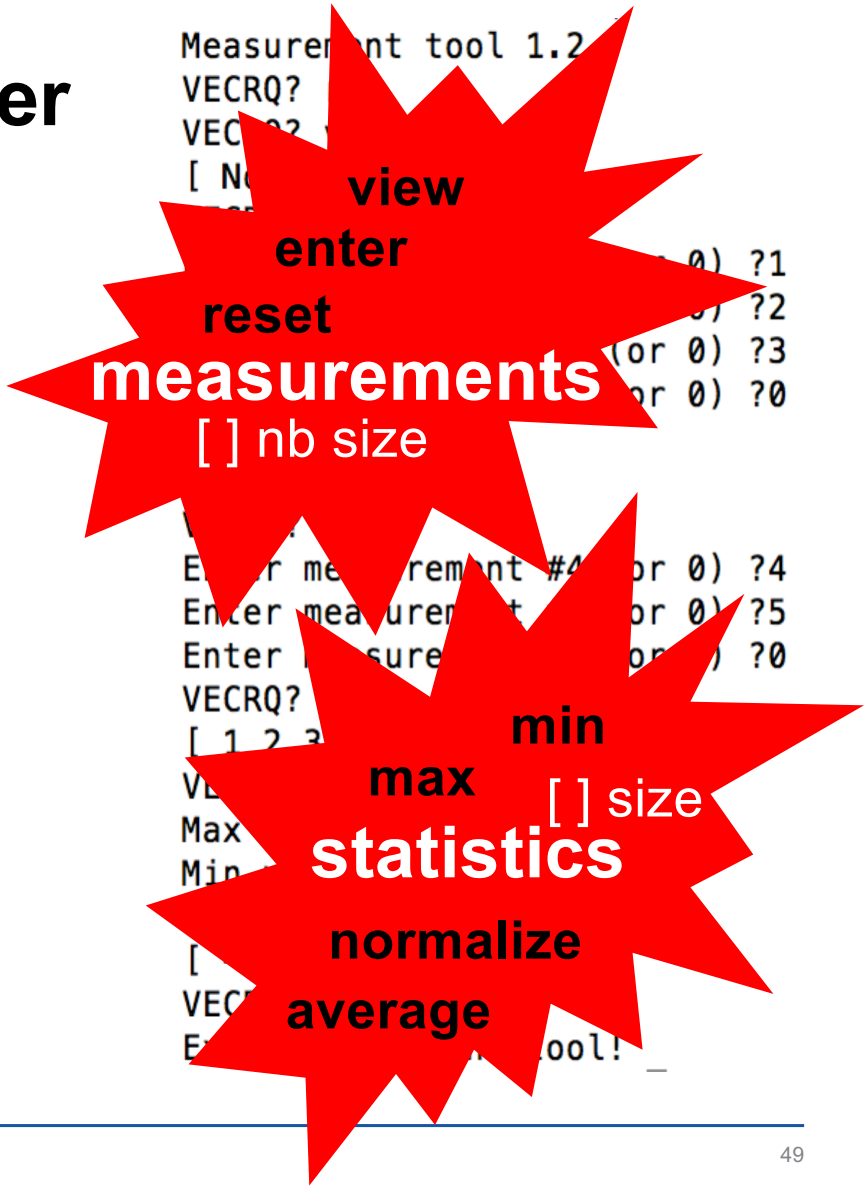
Lab #2: Matriser & Funktioner

Att använda datorprogram för mätdatainsamling och analys är mycket vanligt. Ni ska skriva ett välstrukturerat program där man kan mata in mätvärden och analysera dessa. När programmet startar får användaren upp en meny där denne kan välja:

v (View) - visar vilka mätvärden som finns lagrade
e (Enter) - tillåter användaren att lägga till mätvärden
c (Compute) - presenterar max, min, medelvärde och normaliserade mätvärden
r (Reset) - raderar alla inmatade mätvärden
q (Quit) - avslutar programmet

Användaren ska göra sitt val genom att ange en bokstav…

0) Hum, programmet verkar bestå av tre delar...
...en kommadoavkodare v, e, c, r, q,
...en del för att managera mätvärden,
...och en statistikdel.



…nej, vi tänker top down och gör en systemarkitektur!

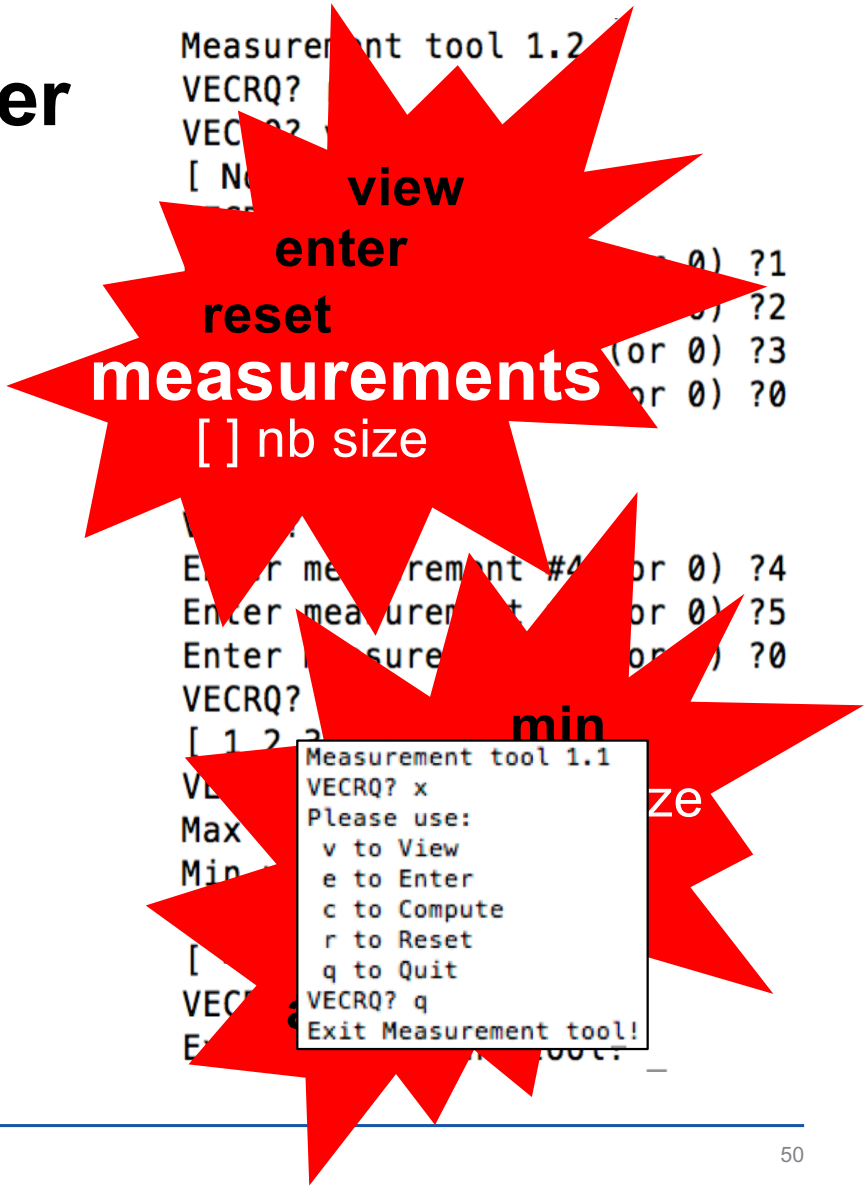
Lab #2: Matriser & Funktioner

Att använda datorprogram för mätdatainsamling och analys är mycket vanligt. Ni ska skriva ett välstrukturerat program där man kan mata in mätvärden och analysera dessa. När programmet startar får användaren upp en meny där denne kan välja:

v (View) - visar vilka mätvärden som finns lagrade
e (Enter) - tillåter användaren att lägga till mätvärden
c (Compute) - presenterar max, min, medelvärde och normaliserade mätvärden
r (Reset) - raderar alla inmatade mätvärden
q (Quit) - avslutar programmet

Användaren ska göra sitt val genom att ange en bokstav…

1) Om jag börjar med kommandoavkodaren blir det lätt att provköra de andra delarna när jag utvecklar dem (samt att programmets intressenter redan nu kan se vad det är jag tror att de beställt)!



…den första kommandoavkodaren kan nog bara q(uit) & h(elp)!

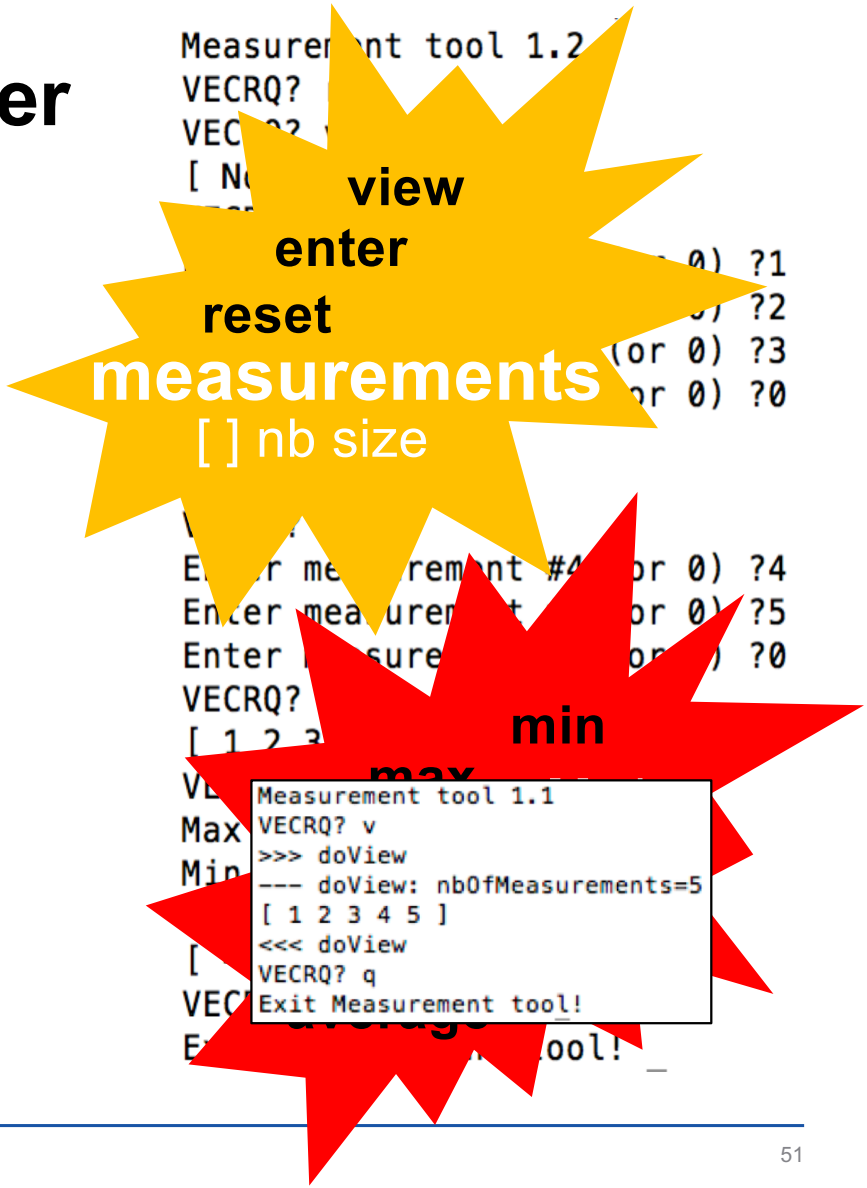
Lab #2: Matriser & Funktioner

Att använda datorprogram för mätdatainsamling och analys är mycket vanligt. Ni ska skriva ett välstrukturerat program där man kan mata in mätvärden och analysera dessa. När programmet startar får användaren upp en meny där denne kan välja:

v (View) - visar vilka mätvärden som finns lagrade
e (Enter) - tillåter användaren att lägga till mätvärden
c (Compute) - presenterar max, min, medelvärde och normaliserade mätvärden
r (Reset) - raderar alla inmatade mätvärden
q (Quit) - avslutar programmet

Användaren ska göra sitt val genom att ange en bokstav…

2) Hum, då är det nog dags att börja fundera på hur mina mätvärden ska lagras, dags för measurements-paketet! Som alltid, skapa nödvändiga variabler med initieringar samt en skriv ut rutin, testa!



…uppdatera kommandoavkodaren så den kan anropa view!

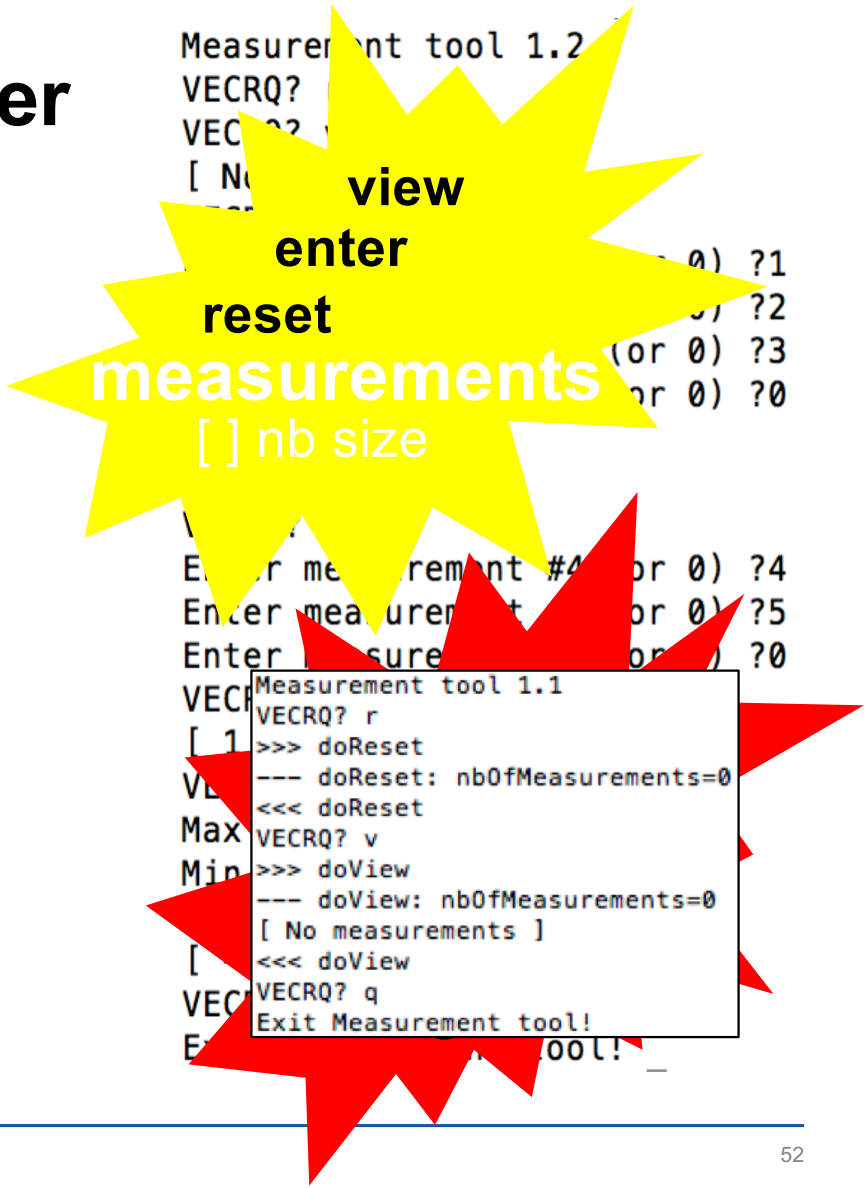
Lab #2: Matriser & Funktioner

Att använda datorprogram för mätdatainsamling och analys är mycket vanligt. Ni ska skriva ett välstrukturerat program där man kan mata in mätvärden och analysera dessa. När programmet startar får användaren upp en meny där denne kan välja:

v (View) - visar vilka mätvärden som finns lagrade
e (Enter) - tillåter användaren att lägga till mätvärden
c (Compute) - presenterar max, min, medelvärde och normaliserade mätvärden
r (Reset) - raderar alla inmatade mätvärden
q (Quit) - avslutar programmet

Användaren ska göra sitt val genom att ange en bokstav…

3) Hum, både enter och reset ska kunna ändrar värdet på antalet lagrade mätvärden, det ska ske via en pekarparameter! Oavsätt det så verkar reset enklare, börjar med den!



…uppdatera kommandoavkodaren så den kan anropa reset!

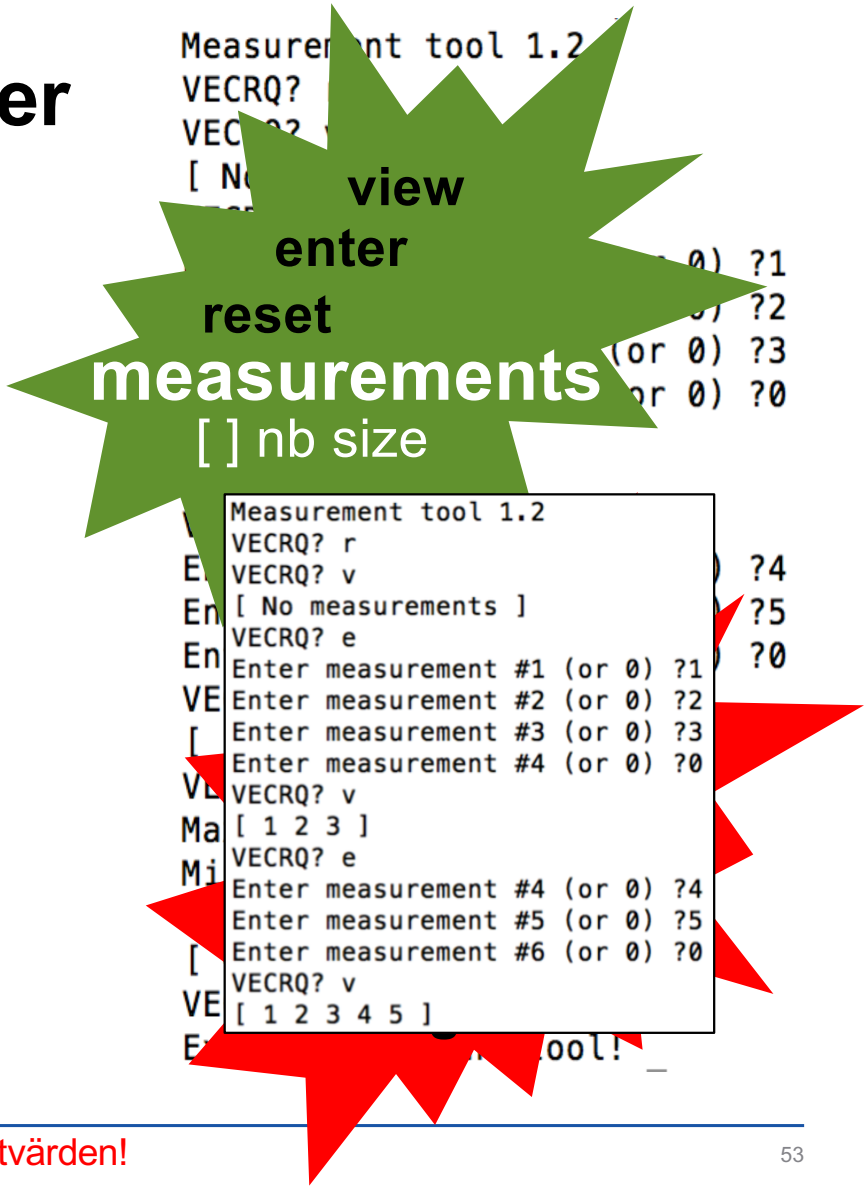
Lab #2: Matriser & Funktioner

Att använda datorprogram för mätdatainsamling och analys är mycket vanligt. Ni ska skriva ett välstrukturerat program där man kan mata in mätvärden och analysera dessa. När programmet startar får användaren upp en meny där denne kan välja:

v (View) - visar vilka mätvärden som finns lagrade
e (Enter) - tillåter användaren att lägga till mätvärden
c (Compute) - presenterar max, min, medelvärde och normaliserade mätvärden
r (Reset) - raderar alla inmatade mätvärden
q (Quit) - avslutar programmet

Användaren ska göra sitt val genom att ange en bokstav…

4) Då är det dags för enter, tänk på att det ska gå att „fortsätta“. Igen, variabel som håller antalet lagrade mätvärden, ska uppdateras via en pekare!



Wow, ett fungerande bibliotek av funktioner som kan hantera mätvärden!

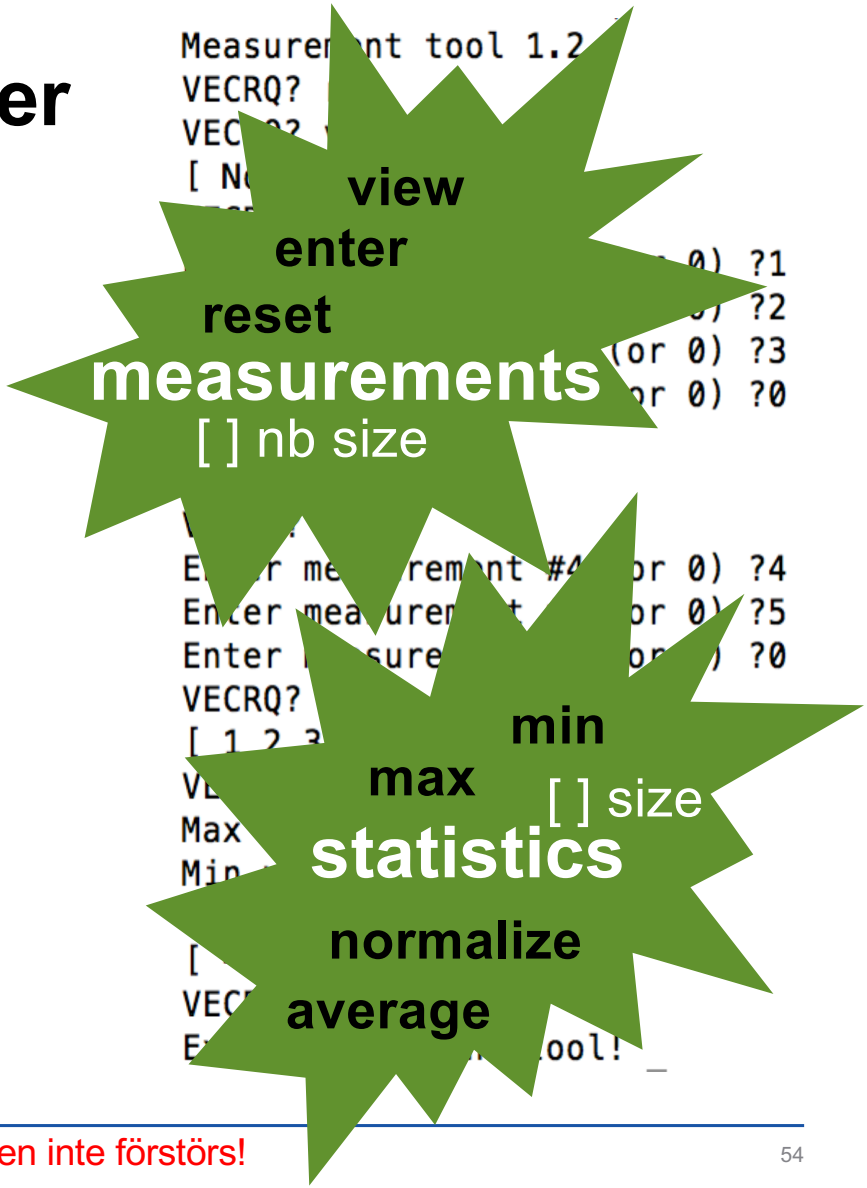
Lab #2: Matriser & Funktioner

Att använda datorprogram för mätdatainsamling och analys är mycket vanligt. Ni ska skriva ett välstrukturerat program där man kan mata in mätvärden och analysera dessa. När programmet startar får användaren upp en meny där denne kan välja:

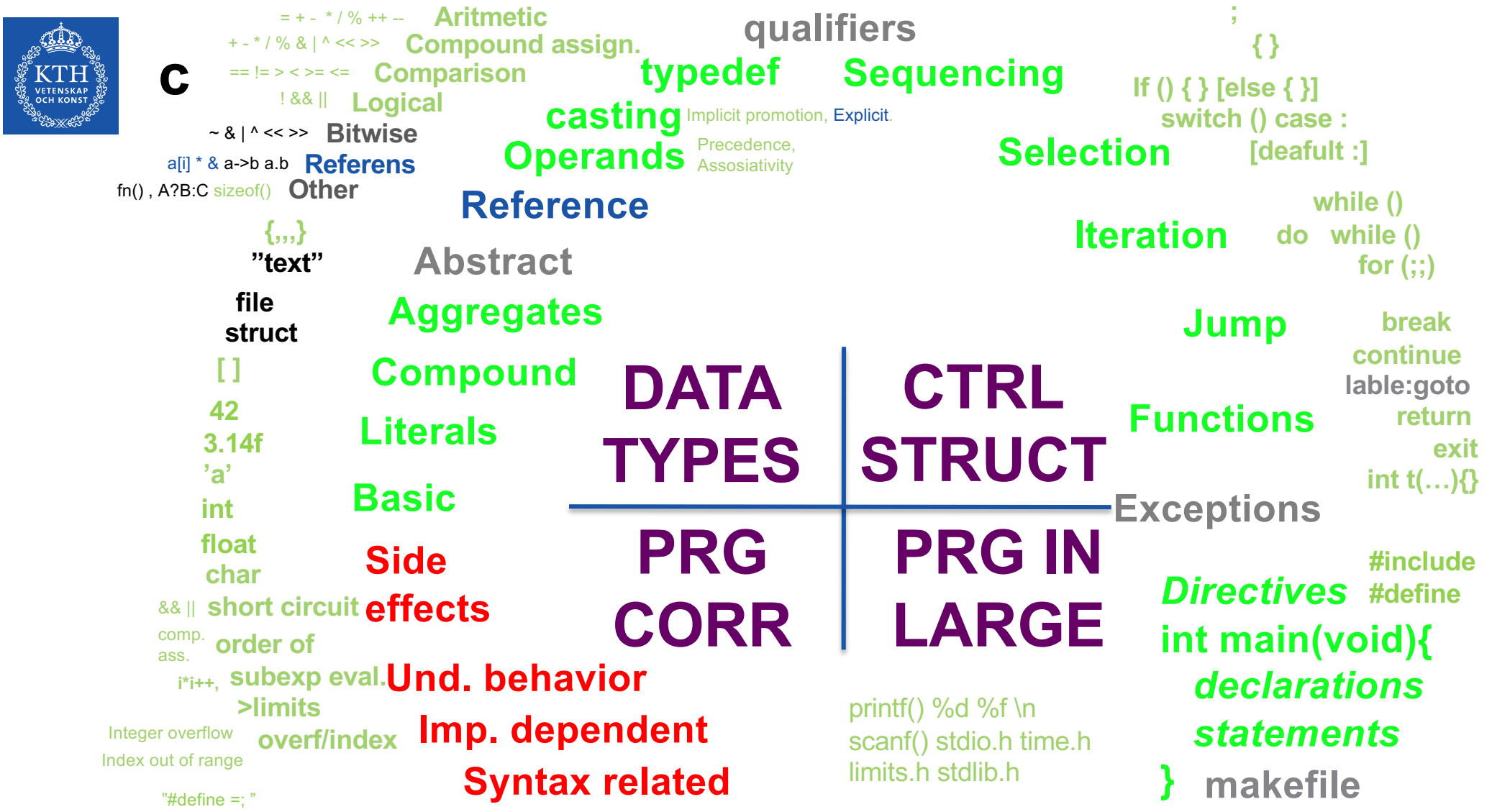
v (View) - visar vilka mätvärden som finns lagrade
e (Enter) - tillåter användaren att lägga till mätvärden
c (Compute) - presenterar max, min, medelvärde och normaliserade mätvärden
r (Reset) - raderar alla inmatade mätvärden
q (Quit) - avslutar programmet

Användaren ska göra sitt val genom att ange en bokstav…

5) Då är det dags för statestikpaketet. Låt antingen main anropa det flera gånger, eller inför en hjälpfunktion. Obs: funktionerna i statestik paketet ska INTE skriva ut något, alla värden ska returneras på något vis!



normalize() ska få en kopia av lagrade mätvärden så informationen inte förstörs!


C
= + - * / % ++ --
Aritmetic
+ - * / % & | ^ << >>
Compound assign.
== != > < >= <=
Comparison
! && ||
Logical
~ & | ^ << >>
Bitwise
a[i] * & a->b a.b
Referens
fn() , A?B:C sizeof()
Other
{,,,}
”text”
file
struct
[ ]
42
3.14f
’a’
int
float
char
Basic
Literals
Compound
Aggregates
Abstract
Reference
qualifiers
typedef
Sequencing
casting
Implicit promotion, Explicit.
Operands
Precedence,
Assosiativity
;
{ }
If () { } [else { }]
switch () case :
[deafult :]
Selection
while ()
do while ()
for (;;)
Iteration
Jump
break
continue
lable:goto
return
exit
int t(…){}
Functions
Exceptions
DATA TYPES
CTRL STRUCT
PRG CORR
PRG IN LARGE
#include
#define
Directives
int main(void){
declarations
statements
}
makefile
&& || short circuit
comp. ass.
order of
i*i++,
subexp eval.
>limits
Integer overflow
Index out of range
overf/index
Und. behavior
Imp. dependent
Syntax related
”#define =; ”
printf() %d %f \n
scanf() stdio.h time.h
limits.h stdlib.h
int/int
/*comment*/ or //

Övn #9: Lab #2

Att använda datorprogram för mätdatainsamling och analys är mycket vanligt. Ni ska skriva ett välstrukturerat program där man kan mata in mätvärden och analysera dessa. När programmet startar får användaren upp en meny där denne kan välja:

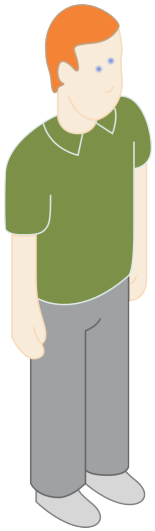
v (View) - visar vilka mätvärden som finns lagrade
e (Enter) - tillåter användaren att lägga till mätvärden
c (Compute) - presenterar max, min, medelvärde och normaliserade mätvärden
r (Reset) - raderar alla inmatade mätvärden
q (Quit) - avslutar programmet

Användaren ska göra sitt val genom att ange en bokstav…

Hela uppgiften finns på Canvas!

```
Measurement tool 1.2
VECRQ? r
VECRQ? v
[ No measurements ]
VECRQ? e
Enter measurement #1 (or 0) ?1
Enter measurement #2 (or 0) ?2
Enter measurement #3 (or 0) ?3
Enter measurement #4 (or 0) ?0
VECRQ? v
[ 1 2 3 ]
VECRQ? e
Enter measurement #4 (or 0) ?4
Enter measurement #5 (or 0) ?5
Enter measurement #6 (or 0) ?0
VECRQ? v
[ 1 2 3 4 5 ]
VECRQ? c
Max value : 5
Min value : 1
Avr value : 3.00
[ -2 -1 0 1 2 ]
VECRQ? q
Exit Measurement tool!
```

Historik



• 150729 0.1	AC	Skapad.
• 150730 0.5	AC	& * Array.
• 150731 1.0	AC	Övning.
• 160917 2.0	AC	Pekare tydligare förklarade.
• 190801 2.0	AC	Omarbetad från grunden, samt anpassad till KTH:s nya profil.